A Project Report on

SCIAM - Smart Collaborative IDE with AI Support and Session Management

Submitted in partial fulfillment of the requirements for the award
of the degree of

Bachelor of Engineering

in

Computer Science and Engineering - Data Science

by

Rishi Bije (22107002)
Dhanraj Bacche (22107056)
Shuban Devanpelli (22107023)
Yash Kamble (22107055)

Under the Guidance of

Ms. Sarala Mary
Mrs. Richa Singh

Department of Computer Science and Engineering - Data Science
A.P. Shah Institute of Technology
G.B. Road, Kasarvadavli, Thane (W) - 400615
UNIVERSITY OF MUMBAI

Academic Year 2025–2026

Approval Sheet

This Project Report entitled *"SCIAM - Smart Collaborative IDE with AI Support and Session Management"* submitted by *Rishi Bije (22107002), Dhanraj Bacche (22107056), Shuban Devanpelli (22107023), and Yash Kamble (22107055)* is approved for the partial fulfillment of the requirement for the award of the degree of *Bachelor of Engineering in Data Science* from *University of Mumbai*.

(Mrs. Richa Singh)
Co-Guide

(Ms. Sarala Mary)
Guide

Dr. Pravin Adivarekar
HOD, Computer Science and Engineering - Data Science

Place: A.P. Shah Institute of Technology, Thane

Date:

CERTIFICATE

This is to certify that the project entitled *"SCIAM-Smart Collaborative IDE with AI Support and Session Management"* submitted by *"Rishi Bije" (22107002),"Yash Kamble" (22107055),"Dhanraj Bacche" (22107056),"Shuban Devanpelli" (22107023)* for the partial fulfillment of the requirement for award of a degree *Bachelor of Engineering* in *Data Science*, to the University of Mumbai, is a bonafide work carried out during academic year 2025-2026.

(Mrs. Richa Singh)
Co-Guide

(Ms. Sarala Mary)
Guide

Dr. Pravin Adivarekar
HOD, CSE(Data Science)

Dr. Uttam D.Kolekar
Principal

External Examiner(s)

1.

2.

Internal Examiner(s)

1.

2.

Place: A. P. Shah Institute of Technology, Thane

Date:

Acknowledgement

We have great pleasure in presenting the report on **SCIAM-Smart Collaborative IDE with AI Support and Session Management** We take this opportunity to express our sincere thanks towards our guide **Ms. Sarala Mary** & Co-Guide **Mrs. Richa Singh** for providing the technical guidelines and suggestions regarding line of work. We would like to express our gratitude towards his constant encouragement, support and guidance through the development of project.

We thank **Dr. Pravin Adivarekar** Head of Department for his encouragement during the progress meeting and for providing guidelines to write this report.

We express our gratitude towards BE project co-ordinators, for being encouraging throughout the course and for their guidance.

We also thank the entire staff of APSIT for their invaluable help rendered during the course of this work. We wish to express our deep gratitude towards all our colleagues of APSIT for their encouragement.

Rishi Bije
(22107002)

Dhanraj Bacche
(22107056)

Shuban Devanpelli
(22107023)

Yash Kamble
(22107055)

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)
Rishi Bije (22107002)

(Signature)
Dhanraj Bacche (22107056)

(Signature)
Shuban Devanpelli (22107023)

(Signature)
Yash Kamble (22107055)

Date:

Abstract

SCIAM (Smart Collaborative IDE with AI Support and Session Management) represents a revolutionary product for collaborative software development with integrated features like real-time code editing, AI development assistance, and smart session management. With AI-based development tools in place, SCIAM offers intelligent assistance in form of suggestions, error detections, and other actions that require complex natural language processing and machine learning techniques. This allows developers to become more efficient at coding and minimize the amount of repetitive tasks that they have to perform. Thanks to smart session management implemented in SCIAM, collaborative coding sessions can be easily tracked and saved for later recovery if necessary. It helps to reduce potential loss in progress, avoid conflicts related to different versions, and ensure transparency of contribution from each participant. Besides code editing in real time, communication and debugging, SCIAM provides session management which helps teams to save their time by reducing the number of tools needed to work collaboratively. Apart from being secure with identity and access management, SCIAM offers valuable analytics tools for visualization and decision-making regarding the status of development process. Due to its AI-driven learning tools, the system is appropriate for both students and teachers, offering customized instruction and feedback when writing codes. Thanks to its cross-platform capabilities, it makes collaboration easy among members even in Windows-based environments. Moreover, due to its plug-ins and APIs, the tool can be easily customized and extended to fit any project requirement. Overall, SCIAM is a future-oriented collaborative platform that connects education, business, and research within software development.

Contents

1 Introduction 1

1.1 Motivation 2

1.2 Problem Statement 2

1.3 Objectives 3

1.4 Scope 3

2 Literature Review 4

3 Project Design 7

3.1 Proposed System 7

3.1.1 Critical Components of System Architecture 8

3.2 Data Flow Diagrams(DFD) 8

3.2.1 DFD Level 0 Diagram 8

3.2.2 DFD Level 1 Diagram 9

3.3 UML Diagrams 10

3.3.1 Activity Diagram 10

3.3.2 Use Case Diagram 11

3.3.3 Sequence Diagram 12

4 Project Implementation 13

4.1 Code Snippets 13

4.2 Steps to access the System 16

4.3 Timeline Sem VIII 18

5 Testing 20

5.1 Software Testing 20

5.2 Functional Testing 21

6 Result and Discussions 22

7 Conclusion 26

8 Future Scope 27

Bibliography 28

Appendices. 29

I. Tools and Technologies Used 29
II. Datasets Used 29
III. Hardware and Software Configuration 29
IV. Code Snippets Overview 30
V. Copyright 31
VI. Research Paper 32

List of Figures

3.1 Proposed System Architecture……...7

3.2 Data Flow Diagrams (DFD)

3.2.1 DFD Level 0 Diagram……...8

3.2.2 DFD Level 1 Diagram……. 9

3.3 UML Diagrams

3.3.1 Activity Diagram……10

3.3.2 Use Case Diagram…….....11

3.3.3 Sequence Diagram…….....12

4.1 Code Snippet

4.1.1 Flask App Initialization……13

4.1.2 Session Creation and Management……..14

4.1.3 Real-Time Code Synchronization……14

4.1.4 Code Execution and Input Handling……....15

4.1.5 WebRTC Signaling for Audio Communication……...15

4.2 Steps to access the system

4.2.1 Get Started…….....16

4.2.2 Collaborative Code Editor……..16

4.2.3 Collaborative Participation……17

4.2.4 Group Chat……17

4.3 Gantt Chart……19

6.1 Performance of Real-Time Synchronization…….....22

6.2 Graph of Communication Stability……....23

6.3 Error Detection……...24

6.4 MOM Quality Comparison……25

8.1 Research Paper Acceptance Mail……...32

List of Tables

2.1 Comparative Analysis of Literature Survey 4

5.2 Functional Testing Table for SCIAM. 21

6.1 Performance of AI Assistance Model 23

List of Abbreviations

SCIAM:	Smart Collaborative IDE with AI Support and Session Management
IDE:	Integrated Development Environment
AI:	Artificial Intelligence
UI:	User Interface
UX:	User Experience
DBMS:	Database Management System
GUI:	Graphical User Interface
ML:	Machine Learning
NLP:	Natural Language Processing
JWT:	JSON Web Token
WS:	Web Socket

Chapter 1

Introduction

There are certain difficulties that may occur during the software development process, such as collaboration, productivity, and code issues. Traditional approaches are based on using several separate tools, managing sessions manually, and collaborating without any system, resulting in duplicates, conflicts in versions, and problems in making decisions. These difficulties can be solved by introducing SCIAM (Smart Collaborative IDE with AI Support and Session Management), which provides a centralized, smart, and collaborative platform for software development.

Due to the possibility of making changes to codes immediately, using AI technologies for receiving helpful information from the application and managing the process automatically, there will be no problem with keeping track of each activity and its impact. This technology will help decrease the number of mistakes in programming by minimizing the amount of time spent on it, and at the same time it will be possible to make decisions effectively and collaboratively.

Moreover, SCIAM has advanced analytics and reporting tools that provide useful insights into team performance, successful completion of tasks, and bottlenecks, thereby enabling organizations to manage their resources effectively and maintain high output quality. By simplifying processes, consolidating all aspects of project activities, and minimizing labor costs, SCIAM enables seamless teamwork and effective software delivery.

Apart from the aforementioned features, SCIAM also facilitates learning and development through artificial intelligence-powered feedback. Therefore, it is suitable for both education purposes and training programs in any organization. SCIAM operates on a cloud computing basis and is highly scalable and accessible. It features a plug-and-play model and API services, providing a highly customized platform for different projects and their unique demands.

In addition to fostering innovation, SCIAM supports fast prototyping, testing, and iterative development in a secure environment. Its user-friendly interface facilitates quick adoption by new users, enabling people from different backgrounds to work together efficiently. Through its combination of intelligent AI-based recommendations and collaborative functionality, SCIAM assists companies in cutting down their development expenses and bringing products to market quickly. Furthermore, SCIAM ensures that there is full and unalterable record keeping of all the activities carried out during the software development process.

1.1 Motivation

There are several key reasons motivating the creation of the proposed system of Smart Collaborative IDE with AI Support and Session Management (SCIAM). The modern software development industry encounters many issues concerning the organization and effectiveness of collaboration within software development teams. Namely, such processes as software coding and debugging can become quite complicated due to poor communication between members of the team.

With the increasing number of projects requiring remote working and collaboration between team members, it is essential to have a comprehensive platform where one could conduct all the processes, ranging from coding to session tracking and analytics. SCIAM is motivated by the need to automate many tedious actions during software development while minimizing the number of mistakes made.Moreover, the project seeks to streamline the process of collaboration and teamwork, providing a convenient tool for professionals as well as students who require some practice in developing software solutions.

1.2 Problem Statement

In today's software development landscape, collaboration, productivity, and maintenance of code quality are major problems among software developers due to their use of a variety of disconnected tools, management of sessions manually, and inconsistent forms of communication. Such practices result in versioning problems, loss of effort, repeated tasks, delayed decision-making processes, and coding mistakes

Remote distributed teams experience even more difficulties as managing contributions, collaboration, and task coordination may prove to be very challenging. The current state-of-the-art IDEs and development platforms do not provide any unified solution combining collaborative capabilities, assistance from artificial intelligence, and automatic management of sessions within one framework.

This leads to inefficiency in work, poor performance of the whole team, and difficulties with project management, making a need for an advanced solution evident.

1.3 Objectives

- **To develop a web-based collaborative IDE that allows 4–5 members to code in real time with host-controlled write permissions:** A web-based IDE should be designed where up to 4 to 5 team members are able to code in real time without any difficulties.

- **To enable built-in chat functionality for seamless communication between team members during sessions:** Integrating a chat application into the IDE platform will make sure that team members are able to discuss and coordinate with each other seamlessly while coding.

- **To create a custom AI module for real-time code correction, suggestions, and debugging support:** An AI module needs to be developed in order to provide code suggestions and fix errors. It would help to improve code writing efficiency and optimization.

- **To implement an AI-powered tracking system to monitor and report individual code contributions to the host:** An AI-based tracking system needs to be designed that can monitor each team member's code contribution and provide detailed reports to the host.

1.4 Scope

- **AI Based Coding:** Next iterations of SCIAM could involve incorporating AI-based coding that includes automatic coding generation, prediction of bugs, and automatic refactoring, thus saving time and improving the process.

- **Multi-Platform Development:** Expansion of the software on mobile, web, and other devices will make it possible for developers to cooperate from any point around the world.

- **DevOps and CI/CD Integration:** Compatibility with DevOps processes and integration with CI/CD tools will allow performing complete project management within the IDE.

- **Education and Practice:** Incorporation of features such as AI-supported tutorials and game-based approach will make SCIAM an educational environment for practicing programming skill.

Chapter 2

Literature Review

The Literature Review discusses several models and frameworks used to overcome problems related to collaborative coding, smart code generation, and real-time code editing by multiple users. This review includes different models ranging from cloud IDEs, AI-enabled code generation and bug fixing utilities, and version control systems. These works point out some crucial shortcomings associated with concurrent editing, version conflicts, and the absence of smooth interaction among communication, debugging, and project management tools. Also, the current models lack the capacity to ensure continuous sessions, support collaborative efforts involving large teams, and provide real-time information on project status.

Taking into consideration all of these problems, it becomes possible to conclude about the gaps that exist in current IDEs, as well as about the creation of a more intelligent as shown in Table 2.1 .

Sr. No	Title	Author(s)	Year	Methodology	Drawback
1	*A Proposal of the Gamification of Programming Learning Assistant System[1]*	Htoo Htoo Sandi Kyaw, Keiichi Kaneko, Nobuo Funabiki	2024	Proposal and prototype design of a gamified Programming Learning Assistant System (PLAS) using game mechanics to improve programming comprehension and engagement for novice students. Implemented in Unity with programming exercises and game-based feedback.	Limited to proposal and initial prototype; lacks largescale testing and empirical validation of effectiveness.
2	*CodeQuest - A Comprehensive Code Learning and Collaboration Platform[2]*	Vaishali Savale, Vedashri Chaudhari, Tejaswini Wanare, Balamurali Nambiar, Manthan Vasant, Sanskruti Umak	2024	Development of an integrated DSA learning platform with a chatbot powered by Llama 27B, real-time code sync, and collaborative code editing using React, Socket.IO, and Express.	Existing systems lack full integration of personalized tutoring and dynamic collaboration; chatbot limitations and scope for broader language/gamification support.

Table 2.1: Comparative Analysis of Literature Survey

Sr. No	Title	Author(s)	Year	Methodology	Drawback
3	*Towards ConflictFree Collaborative Modelling Using VS Code Exten-Sions[3]*	Rijul Saini, Gunter Mussbacher	2021	Proposed a VS Code extension (tColab) for conflict-free real-time collaborative modelling using CRDTs and Live Share for concurrency control and undo handling.	Limited to VS Code; cannot extend to Eclipse Che or Theia due to licensing; semantic conflict handling remains incomplete.
4	*Annotation-based Semantic Conflict Prevention in RealTime Collaborative Programming[4]*	Mingjie Wang, Bicheng Fang, Jinfeng Jiang, Hongfei Fan	2024	Introduced an annotationbased semantic locking mechanism in CoIDEA (IntelliJ IDEA plugin) to prevent editing conflicts and maintain code consistency.	Potential overhead for manual locks; prototype limited to IntelliJ IDEA; requires improved collaborative awareness features.
5	*Collaborative Code Editors – Enabling Real-Time MultiUser Coding and Knowledge Sharing[5]*	Khushwant Virdi, Anup Lal Yadav, Azhar Ashraf Gadoo, Navjot Singh Talwandi	2023	Surveyed collaborative code editors and developed a NodeJS-based prototype using WebSockets and Operational Transformation (OT) for real-time editing.	Latency, synchronization, and conflict-resolution challenges persist; limited scalability for large teams.
6	*Building Temporary Isolated Workspace in RealTime Collaborative Programming Environment[6]*	Jinfeng Jiang, Yuxiang Xie, Bicheng Fang, Mingjie Wang, Hongfei Fan	2023	Proposed RECON approach for maintaining isolated workspaces for compilation/debugging during collaborative programming; implemented in CoIDEA plugin.	Resource overhead from continuous syntactic checking; editing disabled in isolation mode; optimization required.
7	*CodeXchange: Leaping into the Future of AIPowered Code Editing[7]*	Mihir Agrawal, Jatin Goyal, Mradul Goyal, Pratham Sukhija, Jaya Sharma, D. Franklin Vinod	2024	Developed "CodeXchange," an AI-powered editor supporting neural code translation (Python, Java, C++), GPT3-based comment generation, and real-time collaboration using OT and WebSockets.	High computational cost; complex OT conflict resolution; data-sharing security issues; integration challenges with external APIs.
8	*AI Based Code Error Explainer using Gemini Model[8]*	Ankita Amburle, Cheryl Almeida, Oswin Lopes, Nathan Lopes	2024	Built an AI-driven tool for real-time error detection and explanations using Gemini (Bard) Model for Python, C, and Java; provided syntaxhighlighted suggestions.	Limited language support; needs further refinement of AI explanations and multilanguage expansion.

Sr. No	Title	Author(s)	Year	Methodology	Drawback
9	*AI Coding: Learning to Construct Error Correction Codes[9]*	Lingchen Huang, Huazi Zhang, Rong Li, Yiqun Ge, Jun Wang	2020	Proposed AI-driven ECC construction using reinforcement learning (A2C) and genetic algorithms; evaluated decoding performance via simulations.	High computational demand; offline training limits adaptability; restricted to simulated datasets.
10	*An Empirical Study on the Usage of Transformer Models for Code Com-Pletion[10]*	Matteo Ciniselli, Nathan Cooper, Luca Pascarella, Antonio Mastropaolo, Emad Aghajani, Denys Poshyvanyk, Massimiliano Di Penta, Gabriele Bavota	2021	Evaluated Transformer-based models (RoBERTa, T5) for code completion using Java/Android datasets, comparing performance with n-gram baselines.	Limited language scope; high resource requirements; accuracy decreases for larger code blocks.
11	*Code Generation Using Self-Interactive Assistant[11]*	Zixiao Zhao, Jing Sun, Cheng-Hao Cai, Zhiyuan Wei	2024	Developed "SelfCoder," a GPT-4-based interactive code generation framework combining knowledge retrieval and code interpreter modules for iterative optimization.	Struggles with ambiguous inputs, high operational cost, and fixed API parameter limitations.
12	*The Development of a Collaborative Learning Tool for AI Education: Corobolab[12]*	Haibin Lai, Peng Peng, Yuhui Lv, Peiwei Cai, Qiongxiong Ma, Zhun Zhang	2025	Developed "Corobolab," a collaborative AI-learning environment integrating software–hardware kits for K-12 education with real-time programming and networked feedback.	Limited UI clarity; network latency issues; currently tailored to K-12 education; needs adaptation for advanced curricula.
13	*LiteDoc: Make Collaborative Editing Fast, Scalable, and Robust[13]*	Saptaparni Kumar, Haochen Pan, Roger Wang, Lewis Tseng	2020	Proposed LiteDoc – a sectionbased editor using atomic registers and writer locks to improve throughput and robustness in collaborative editing.	Limited features compared to mainstream editors; only one writer per section; UI under development.
14	*AI-Driven Web Application Automated Testing and Code Optimization[14]*	Junbang Ma	2025	Implemented deep-learningbased framework combining LSTM for test generation, Q-learning for adaptive execution, and CNN for code quality evaluation.	High resource consumption; limited multi-language support; complex adaptation to evolving codebases.
15	*Multiple Authors Identification from Source Code Using Deep Learning Model[15]*	Abdul Mannan Omi, Monir Hossain, Md Nahidul Islam, Tanni Mittra	2021	Used deep learning with AST + code2seq features to identify authorship segments across 3021 Java files; achieved 96.7% accuracy with DNN.	Restricted to Java; scalability issues for larger datasets; reliant on AST parser quality.

Chapter 3

Project Design

3.1 Proposed System

This figure shows the architectural structure of the SCIAM platform used in collaborative software development. This platform focuses on some of the most important features, including collaborative IDE, artificial intelligence-based code assistant, session management, communication module, and report generation. Figure 3.1 demonstrates how these features enable collaboration, effective code assistance, secure access, and contribution tracking.

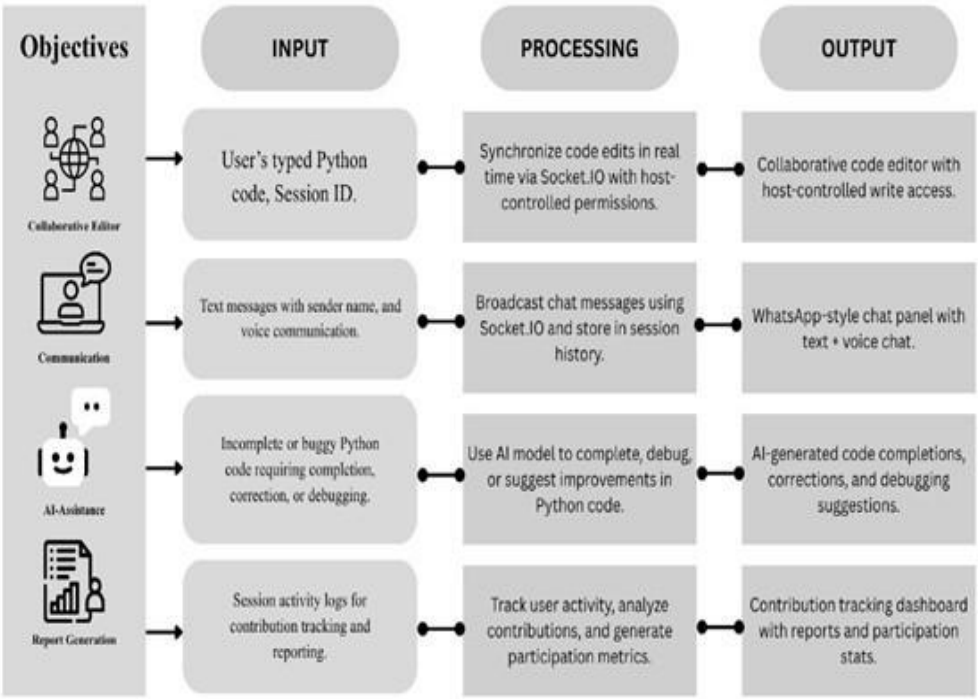


Figure 3.1: Proposed System Architecture

3.1.1 Critical Components of System Architecture

A Collaborative IDE Environment

IDE is the single platform where multiple programmers can write code at once. This platform consists of all necessary components that include editors, version control system, and debuggers. The IDE contains an integrated chat feature and other communication systems that help team members communicate instantly while writing code without using any external means for communication.

B AI-Powered Code Assistant

This module leverages NLP and machine learning models to provide smart code suggestions, error detection, and real-time debugging support. It enhances coding efficiency by reducing repetitive tasks and preventing common programming error

C Report Generation

The software automatically tracks collaborative sessions, ensuring that all the contribution of an individual during the session is stored and analyzed.

D Communication

This module allows the praticipants to communicate with each other by text messages and voice enabled communication.

3.2 Data Flow Diagrams(DFD)

3.2.1 DFD Level 0 Diagram

DFD Level 0 is a representation of the process flow that happens inside the SCIAM - Smart Collaborative IDE with AI Support and Session Management. It gives a view of the system structure showing all its components and interaction with each other such as host, team members, collaborative editor, AI component, and database. This diagram shows how users use the system for coding and communications purposes through the use of artificial intelligence as illustrated in figure 3.2.1.

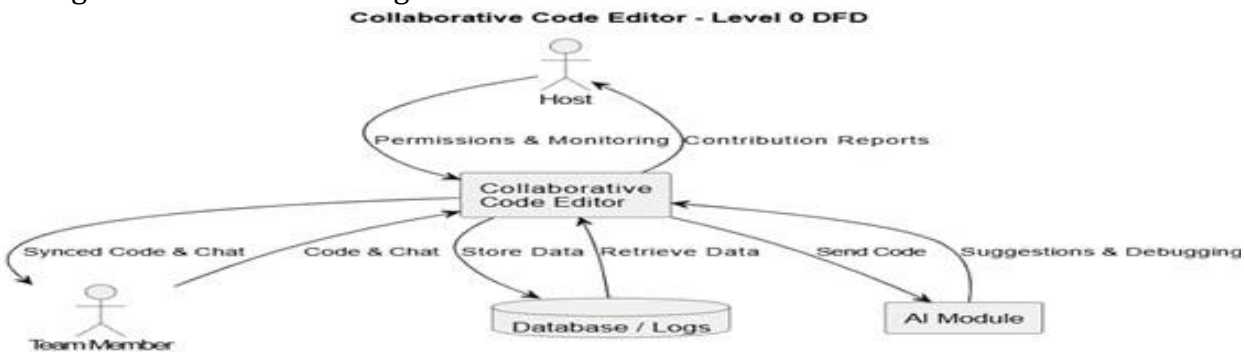


Figure 3.2.1 : DFD Level 0 Diagram

In Figure 3.2.1, the information flow commences when the Host and Team Members use the Collaborative Code Editor in order to join the session for collaboration. The Host sets the rules and tracks the activities of other participants, while Team Members contribute to the session by writing and editing code and messages. The Collaborative Code Editor interfaces with the Database/Logs in order to save and process data such as session history, messages, and code versions. At the same time, the AI Module interfaces with the code provided by the Collaborative Code Editor in order to offer its suggestions and advice regarding coding.

3.2.2 DFD Level 1 Diagram

The Level 1 DFD gives an elaborate description of the internal processes involved in the SCIAM – Smart Collaborative IDE with AI Support and Session Management. The interaction between different components including Host Control, Real-Time Collaboration, AI Support, Database, and Chat System can be visualized through the diagram given below as figure 3.2.2.

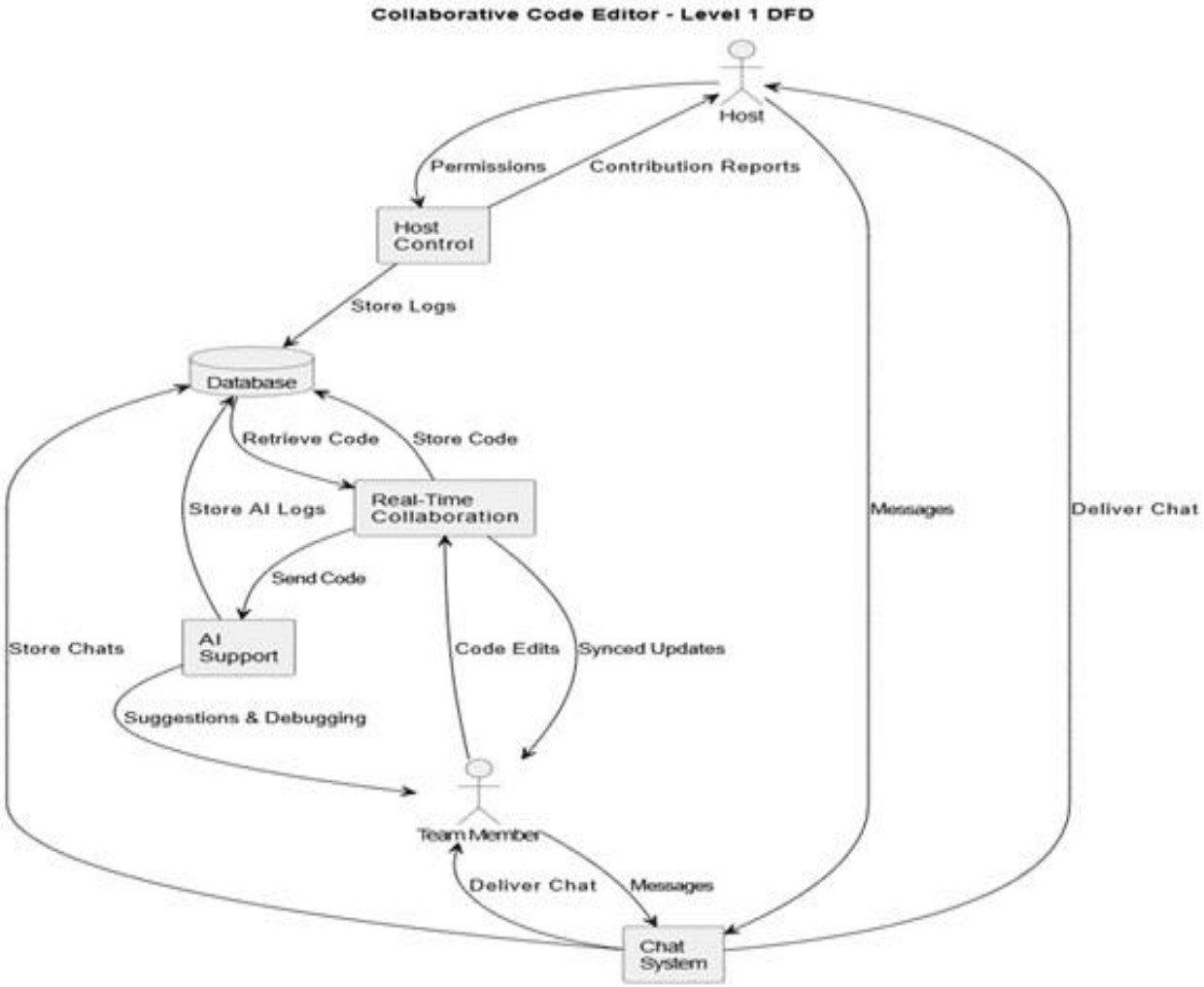


Figure 3.2.2 : DFD Level 1 Diagram

In Figure 3.2.2, the Host manages session management using the Host Control module that gives users the authority to work on projects. The Real-Time Collaboration module is responsible for code editing and synchronization between the Host and the Team Members. The module will also help in improving the coding process through analysis and debugging, thereby creating optimized logic. The AI Support module will store necessary logs within the Database. The Chat module will facilitate seamless communication among the participants through messaging systems within the system. All information such as versions of code, AI logs, and chat conversations will be stored in the Database.

3.3 UML Diagrams

3.3.1 Activity Diagram

This activity diagram explains the process of activities that occur inside the SCIAM system. The activity diagram depicts how the user starts a session, chooses his role to either be a Team Member or a Host, and engages himself in collaborative programming as illustrated in figure 3.3.1 below.

Figure 3.3.1 : Activity Diagram

An activity diagram in UML is used to demonstrate how actions flow logically within the SCIAM system. The first action that occurs is the initialization of the session, then the role selection follows, which determines whether the user will be creating/ managing their sessions as the Host, or joining any ongoing session as a member of the Team. The two options end up with facilitating real-time collaboration through coding together while receiving AI support in suggesting intelligent changes in code. The Session Manager continuously monitors all the code changes and maintains its version control and synchronization. After that, the code gets saved and synchronized on the Database. At last, a contribution report is produced indicating the contributions from all participants before ending the session.

3.3.2 Use Case Diagram

The Use case diagram for SCIAM is depicted here and shows the primary functions of the system. There are only two kinds of actors involved in this system. These include Host/Administrator and Developer, interacting with each other using the SCIAM platform. Firstly, registration and authentication are required by all users who want to gain access to SCIAM platform. Next, the Host/ Administrator will be privileged to initiate or manage any coding session. On the other hand, developers will be allowed to join an already initiated coding session to collaborate on the platform as shown in Figure 3.3.2.

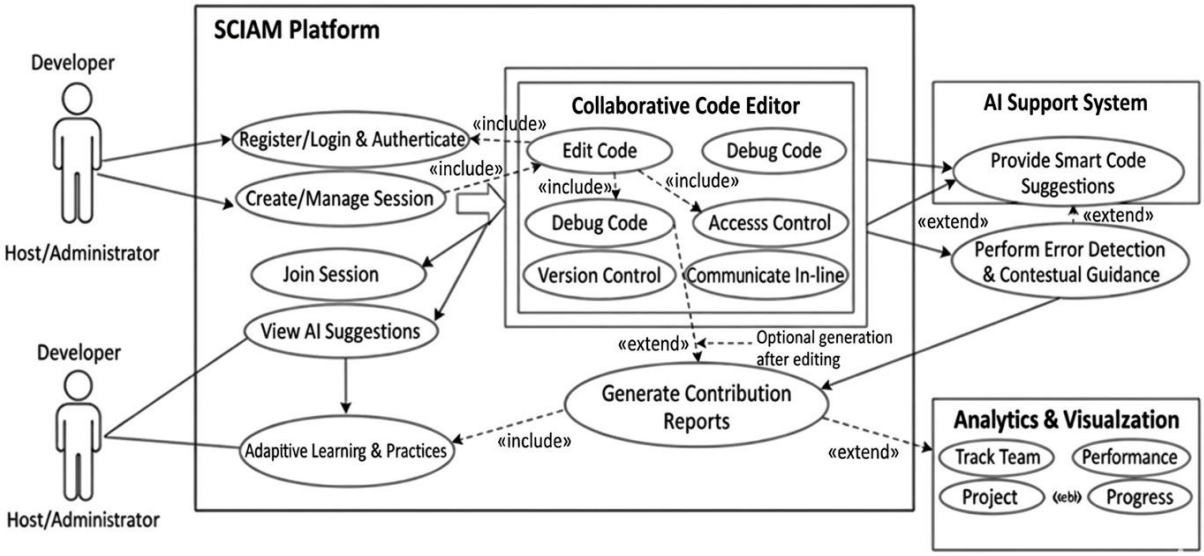


Figure 3.3.2 : Use Case Diagram

The central element of this framework is the Collaborative Code Editor, acting as the primary environment for developers. In this module, users can edit, debug, and manage the version control of their projects. Moreover, the module allows users to communicate and provide feedback directly in the code itself. The AI Support System is integrated within the code editor, offering suggestions for writing codes, performing automatic debugging, and detecting errors within the code context. This makes the coding process more efficient and saves time on tedious coding tasks. Finally, the system provides access control measures to guarantee seamless coordination among various participants in a single project.

Moreover, the system integrates an Analytics and Visualization module to monitor the performance and progress of team members. It also tracks contributions from both the hosts and developers and generates reports that allow a transparent evaluation of the productivity levels of the involved parties. Similarly, the Adaptive Learning and Practice module is another essential component of the IDE framework. It utilizes artificial intelligence to provide feedback and recommend learning skills for the user. Consequently, SCIAM is an effective and efficient IDE framework that promotes collaboration, accurate coding, and learning.

3.3.3 Sequence Diagram

The SCIAM – Smart Collaborative IDE with AI Support and Session Management Sequence Diagram is a diagram depicting how users and system modules work together with AI modules to create an integrated and collaborative coding experience for developers. The process starts when a developer logs into the system, and the login process ensures that they gain access to their accounts via a secure session created using the help of the Session Manager Module. After the login process, the developer then joins a session either by creating one or joining others using the Collaborative Editor Module, which becomes the main platform where all participants are involved.

In the collaborative phase, the Collaboration Server enables real-time synchronization of code updates made by all participants connected to the same session. This way, all team members see an updated copy of the code and work more efficiently together. At the same time, the AI Assistant Module works in the background and analyzes the actions performed by the developer. This module detects possible coding mistakes and generates useful suggestions for the code. Using such an innovative AI approach can significantly increase developers' productivity and accelerate debugging processes. The simultaneous functioning of the Collaborative Editor, AI Support System, and Session Manager guarantees that all changes and updates are instantly reflected in all copies of code without any conflicts as is demonstrated in Figure 3.3.3 below.

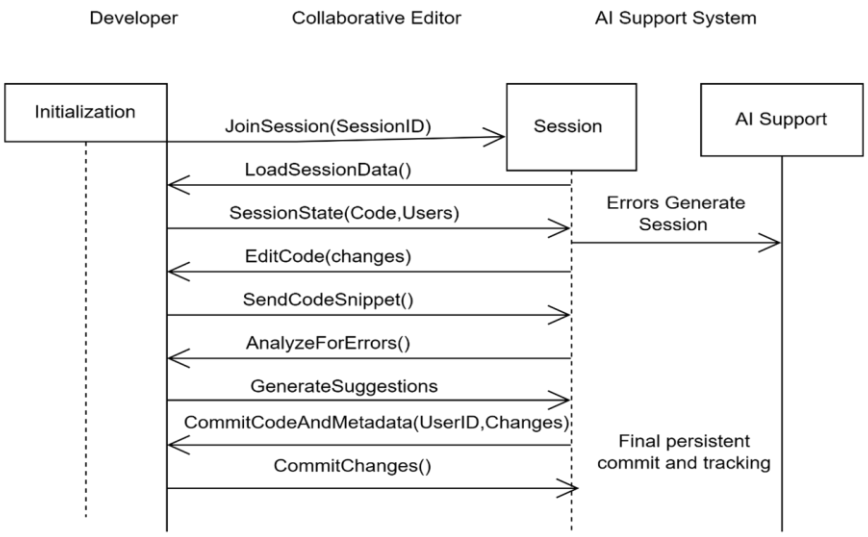


Figure 3.3.3 : Sequence Diagram

The third and last stage involves safeguarding data consistency and logging the contributions made by each participant. The Session Manager will initiate the process of automatically saving data into the database to ensure everything done is logged consistently. At the end of the session, the developer's work is committed using CommitChanges(), which records the code and relevant metadata.

Chapter 4

Project Implementation

The implementation phase of the project deals with the actual development and realization of the designed architectural structure according to the objectives set by the project. In this phase, the focus was laid on building some important features like backend development, UI building, and live data processing. Important consideration was paid to integration of various technologies and optimization of the system. Some critical parts of the system were tested repeatedly in order to check their proper functioning, as well as security and robustness of the solution developed. Besides, some automated documentation and analytics were implemented in order to get useful insights from system performance and user behavior.

4.1 Code Snippets

In this segment, the focus will be on some sample code excerpts from SCIAM. This will help us see how vital functionalities were incorporated in the system. These code snippets showcase the logic and features of several modules used in the SCIAM system. They include but are not limited to sessions, AI assistant, and collaboration functionalities. These code samples show how each module works together to provide seamless integration of the system. They also give insight into the practicalities involved in implementing several features of SCIAM. In addition, they show how Flask-SocketIO helps in providing live communication between different users. On the other hand, WebRTC ensures that peer-to-peer communication through voice channels is efficient.

```
import os
import queue
import select
import sys
import threading
import uuid
import tempfile
import subprocess
import time
import shutil
import re
from datetime import datetime
from flask import Flask, render_template, request, jsonify, redirect, url_for, flash, session as flask_session
import flask
from flask_socketio import SocketIO, emit, join_room
from werkzeug.utils import secure_filename
from werkzeug.security import generate_password_hash, check_password_hash
from functools import wraps
import pymysql
from pymysql.cursors import DictCursor


# Flask app initialization
app = Flask(__name__)
app.config['SECRET_KEY'] = os.environ.get('SECRET_KEY', 'dev_secret_key')
```

Figure 4.1.1: Flask App Initialization

The below Python code is that of a Flask web app initialization routine, where first the imports from the Flask class and various utility modules like rendering templates and request

module, along with other extensions including flask socketio, which provides WebSocket capabilities to the app, are included. Then, the app itself, app = Flask(name), is created. Here, the secret key for the Flask application is assigned by trying to fetch it from an environment variable first, and then by using a development key “dev secret key,” as seen in Fig. 4.1.1.

```
@app.route("/create_session", methods=["POST"])
def create_session():
    session_id = str(uuid.uuid4())[:8]
    sessions[session_id] = {
        # Multi-file structure
        "files": {
            "main.py": "# Welcome to SIREN Collaborative Editor\n# Start coding in Python...\nprint('Hello, World!')"
        },
        "active_file": "main.py",
        "participants": {},
        "host_id": None,
        "writer_id": None,
        "chat_messages": []  # Added for chat
    }
    print(f"🎉 New session created: {session_id}")
    return jsonify({"session_id": session_id})
```

Figure 4.1.2: Session Creation and Management

This process is responsible for initiating the new coding session and assigning a unique ID to it. It creates the initial settings for file structure, active file, users, and chat messages. The information related to the session will be stored temporarily in the memory and distributed to all the users connected in the same session (see Figure 4.1.2).

```
@socketio.on("code_change")
def handle_code_change(data):
    session_id = data.get("session_id")
    content = data.get("content", "")
    file_name = data.get("file_name") or data.get("file") or None
    sid = request.sid

    if session_id in sessions:
        session = sessions[session_id]
        # Only allow the current writer to make changes
        if session["writer_id"] == sid:
            target_file = file_name or session.get("active_file", "main.py")
            if "files" not in session:
                session["files"] = {}
            if target_file not in session["files"]:
                session["files"][target_file] = ""
            session["files"][target_file] = content
            emit("code_update", {"file_name": target_file, "content": content}, room=session_id, include_self=False)
            author = session['participants'].get(sid, {}).get('name', 'unknown')
            print(f"📝 Code updated in {target_file} by {author} in session {session_id}")
```

Figure 4.1.3: Real-Time Code Synchronization

This component of the backend supports real-time synchronization among different users. In the event that any user (writer) makes changes in the code, the information is instantly sent to other users within the same session. The communication process is made possible by the use of Flask-SocketIO, which allows for instant data exchange and versioning within sessions.

```python
@app.route("/run_code", methods=["POST"])
def run_code():
    data = request.get_json()
    code = data.get("code", "")
    session_id = data.get("session_id")
    user_input = data.get("user_input", "")
    process_id = data.get("process_id")
    # Optional parameters
    timeout_seconds = int(data.get("timeout_seconds", 120))
    timeout_seconds = max(1, min(timeout_seconds, 300))  # allow 1..300s
    use_docker = bool(data.get("use_docker", False))
    docker_packages = data.get("docker_packages", [])  # e.g. ["numpy","pandas"]

    # If this is providing input to an existing process
    if process_id and user_input:
        if process_id in input_queues:
            input_queues[process_id].put(user_input + "\n")
            return jsonify({"status": "input_sent", "message": "Input sent to process"})
        else:
            return jsonify({"status": "error", "message": "Process not found or completed"})
    try:
        # Better detection for interactive code:
        # strip triple-quoted / quoted strings and comments first, then search for real input(...) or sys.stdin
        def _strip_strings_and_comments(s):
            s = re.sub(r('"""[\s\S]*?"""|\'\'\'[\s\S]*?\'\'\')', '', s, flags=re.S)  # triple-quoted
            s = re.sub(r('"[\s\S]*?"|\'[\s\S]*?\')', '', s, flags=re.S)               # single/double quoted
            s = re.sub(r'#[^\n]*', '', s)                                             # line comments
            return s

        cleaned = _strip_strings_and_comments(code or "")
        code_needs_input = bool(re.search(r'\binput\s*\(', cleaned)) or bool(re.search(r'\bsys\.stdin\b', cleaned))
        # Create a unique process ID for this execution
        process_id = str(uuid.uuid4())
        # Create input queue for this process (keep queue present to avoid race conditions)
        input_queues[process_id] = queue.Queue()
        process_needs_input[process_id] = code_needs_input
```

Figure 4.1.4: Code Execution and Input Handling

The code here allows for the execution of user-generated code on the server using Flask. This code allows capturing of the Python script generated by the user and executing it safely and managing any dynamic user input, if there is any. The code also determines if the input has any interactive commands such as the input() command and streams the output back to the client.

```python
# WebRTC signaling handlers
@socketio.on("webrtc_offer")
def handle_webrtc_offer(data):
    target_sid = data.get("target")
    offer = data.get("sdp")
    if target_sid:
        emit("webrtc_offer", {
            "sdp": offer,
            "sid": request.sid
        }, to=target_sid)

@socketio.on("webrtc_answer")
def handle_webrtc_answer(data):
    target_sid = data.get("target")
    answer = data.get("sdp")
    if target_sid:
        emit("webrtc_answer", {
            "sdp": answer,
            "sid": request.sid
        }, to=target_sid)

@socketio.on("webrtc_ice_candidate")
def handle_webrtc_ice_candidate(data):
    target_sid = data.get("target")
    candidate = data.get("candidate")
    if target_sid:
        emit("webrtc_ice_candidate", {
            "candidate": candidate,
            "sid": request.sid
        }, to=target_sid)
```

Figure 4.1.5: Critical Components of System Architecture

The sample code shows WebRTC signaling functions that facilitate direct voice communication between users. This takes care of the offer-answer mechanism and ICE candidate exchanges needed to create a secured connection for media streaming. This way, SCIAM guarantees direct voice communication between users during the same session while they code, as depicted in Figure 4.1.5.

4.2 Steps to access the System

Step 1: Get Started

This is the initial phase of the SCIAM process wherein the User may initiate a session. The method through which a user enters into a session can be done either by starting a new session or joining an ongoing session, as illustrated in Figure 4.2.1.

Figure 4.2.1 : Get Started

Step 2: Collaborative Code Editor

This is a collaborative editor used in editing codes of python or java program and several files may be edited simultaneously as well. Only one editor may edit at any one time but the host may give certain people access as illustrated in Figure 4.2.2 below.

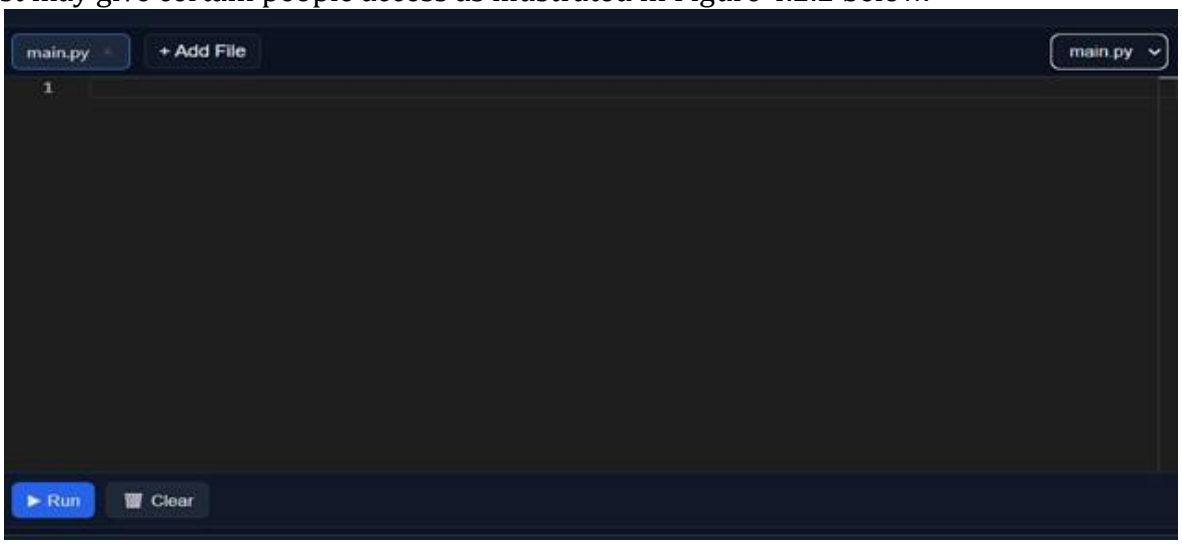


Figure 4.2.2 : Collaborative Code Editor

Step 3: Collaborative Participation

It is a web-based collaborative IDE that allows multiple members to code in real time with host-controlled write permissions as shown in Figure 4.2.3 .

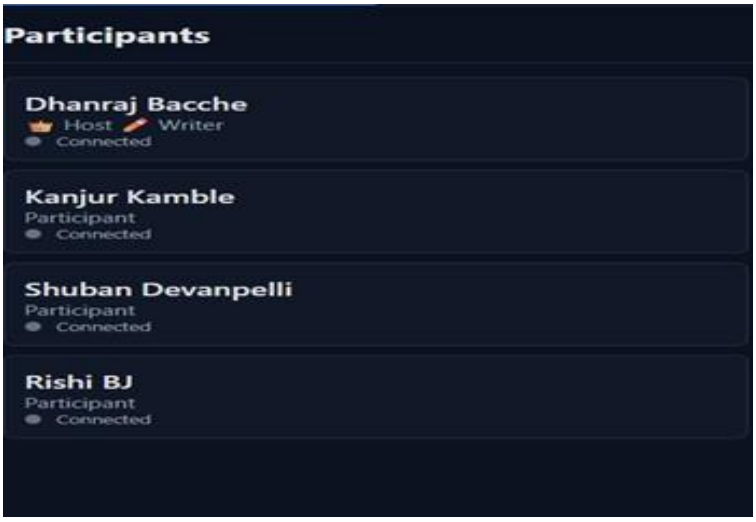


Figure 4.2.3 : Collaborative Participation.

Step 4: Group Chat

The step helps achieve integrated chat capability for smooth communication among team members during the session. Chat capability within the IDE helps team members communicate instantly without moving away from the platform as illustrated in Figure 4.2.4.

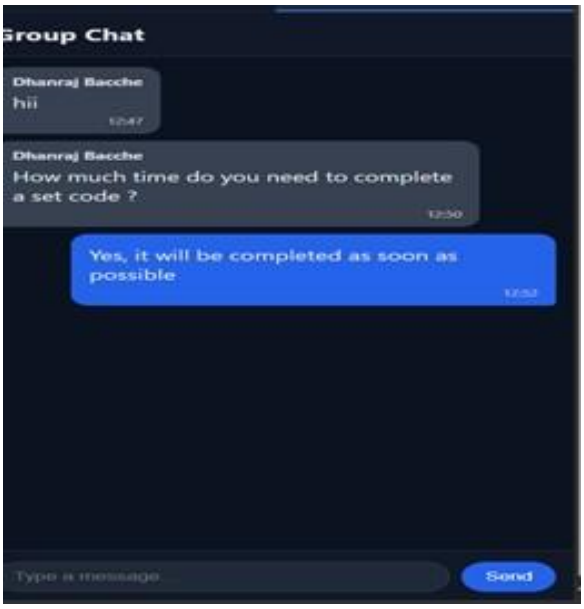


Figure 4.2.4 : Group Chat

4.3 Timeline Sem VIII

In order to deal with increasingly difficult software development projects, proper project management and milestone management are imperative. The following part provides a description of the project timeline and important milestones in Semester VII, starting from the conception stage, moving onto the design, implementation, and testing stages of the project.

It was essential that there be tight cooperation among all project members, along with strict adherence to the deadline. The Gantt chart provides visual information about the schedule of tasks. It demonstrates the relationship between each task and the entire project process and shows the interrelationship of tasks, which is vital when scheduling overlapping tasks.

The timeline of the group for the whole semester was quite detailed and involved project scope definition and research at the start of the process, followed by coding, integration, and testing stages. The group conducted regular evaluations and updates on the state of the project in order to be able to prioritize tasks depending on their importance.

Also, the Gantt chart indicates the percentage of the work done on each task, thus giving an insight into the progress of the project at various stages. The group was able to overcome common problems associated with big development projects, such as task coordination and deadlines. Generally, the timeline illustrates the entire process of developing the project, and is reflected in Figure 4.3.

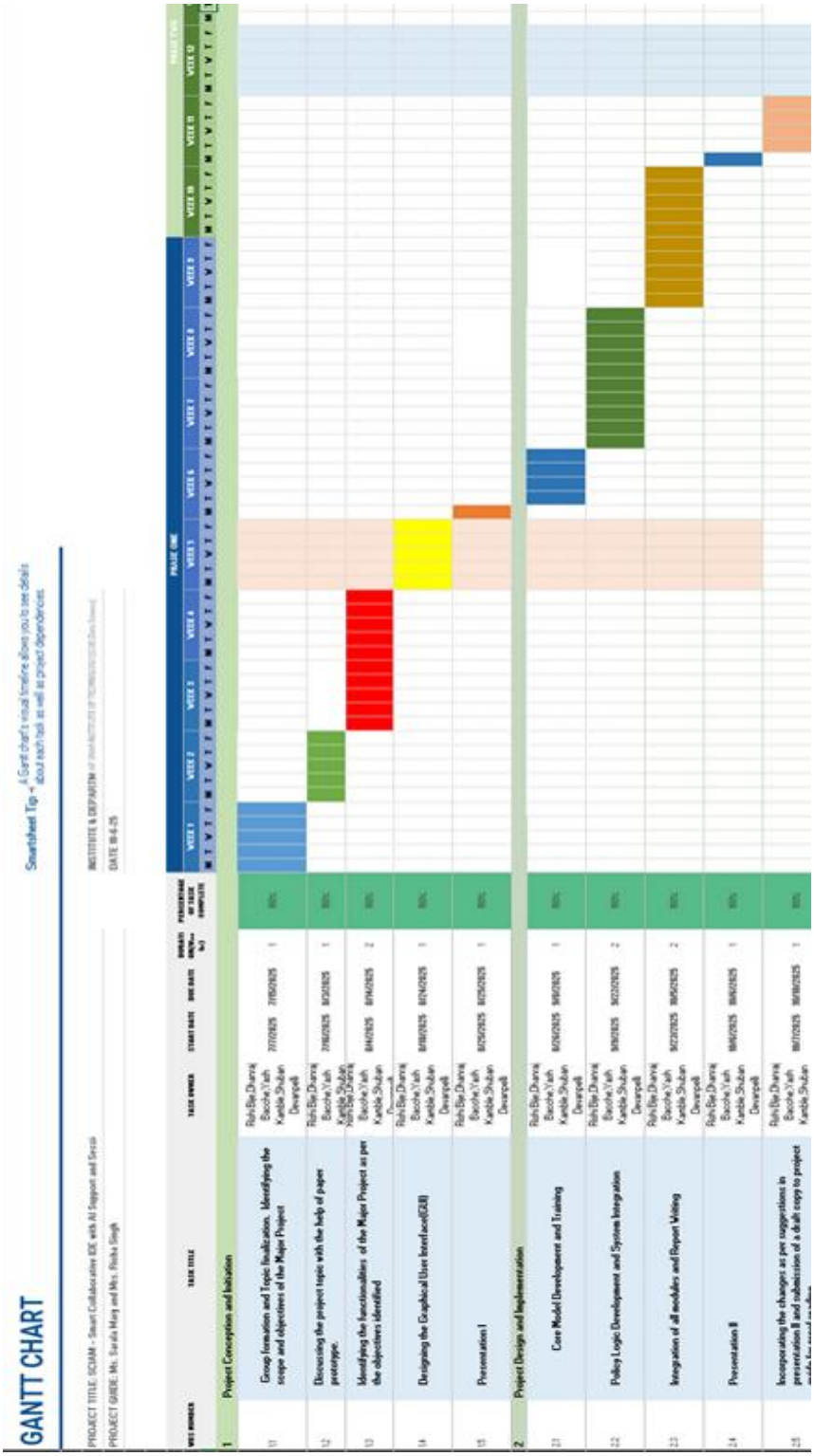


Figure 4.3: Gant Chart

Chapter 5

Testing

5.1 Software Testing

The SCIAM - Smart Collaborative IDE with AI support and Session Management has been subjected to rigorous testing to ensure that it is functionally correct, performant, integrated, and stable in both single-user mode and collaboration mode.

The various components of the system including the AI Assistance Engine (based on NLP Models), Real-time Collaboration Engine, Session Management Engine, Flask Server backend connected to the database, and User Interface have been extensively tested for their accuracy, responsiveness, and interoperability.

The AI Assistance Engine has been tested using multiple code suggestions and queries from users. The AI Assistance Engine has been tested for its accuracy in responding to code suggestion requests.

The Real-Time Collaboration Module has been tested for synchronization efficiency during multiple user sessions. Various scenarios such as concurrent coding and messaging have been tested to ensure minimal latency and seamless merge process without any conflicts among the collaborating users.

The Session Management system has been validated for the purpose of ensuring security, authentication, continuous functioning, and recovery from interruption or disconnection. This testing is especially important when it comes to managing user state management and versioning.

Flask Backend and Database Layer have been assessed for purposes of ensuring proper data transmission, logging and storing project files, artificial intelligence interaction records, and other collaboration data.

The User Interface has been tested for the purpose of determining usability, responsiveness, and compatibility across multiple devices.

Testing Objectives

- Test the functionality of code suggestions provided by the AI, real-time collaboration features, and session handling components.
- Maintain smooth interaction between the AI, Flask back-end, and the collaboration services.
- Confirm that session handling is secure and persistent for several simultaneous users.
- Test performance factors including latency and throughput during live coding.
- Perform checks to confirm that data integrity is maintained during project saving, code versioning, and synchronization events.
- Conduct tests on the system's stability under simultaneous requests for AI operations and collaboration.

5.2 Functional Testing

For this purpose, the proposed project follows the Black Box Testing Technique using Functional Testing. The reason being that it will be ensured that each module in the SCIAM – Smart Collaborative IDE with AI Support and Session Management operates as per its functional specifications. Black box testing focuses on assessing the behavior of the software in terms of receiving various user inputs, collaboration, and AI-supported coding queries. Furthermore, the correct outputs generated as a result of these queries would be examined such as code suggestions in real-time, accuracy of synchronization, and continuous session management. Black Box Functional Testing is particularly appropriate for our project as it guarantees complete functionality, usability, and integration of all the software components.

Test Case ID	Description	Expected Result	Status
TC-01	Session created by host	Unique session ID displayed	Passed
TC-02	Participant joins session	Participant joins session	Passed
TC-03	Real-time collaboration	Multiple users edit concurrently	Passed
TC-04	Voice chat enabled	Clear audio transmission	Passed
TC-05	AI suggests code completion	Relevant code suggestions displayed in real time	Passed
TC-06	Generate contribution report	Detailed report available for host	Passed

Table 5.2 : Functional Testing Table for SCIAM

Chapter 6

Result and Discussions

SCIAM – Smart Collaborative IDE with AI Support and Session Management has been found very effective in meeting its aim of creating an intelligent, real-time, and collaborative programming environment. This has been possible through the inclusion of modules like artificial intelligence-based code support, real-time collaboration of editing, and session management.

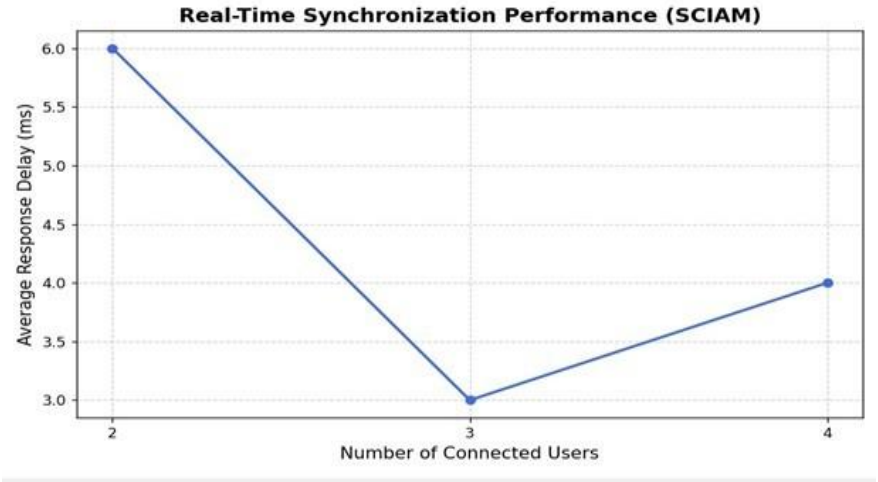


Figure 6.1: Performance of Real-Time Synchronization

The above figure 6.1 is the latency of SCIAM Collaborative Code Editor in terms of the number of participants involved in the collaboration process in real-time. The y-axis indicates the average latency (in milliseconds) of the message while the x-axis represents the number of participants connected in the same collaborative code editor session. As indicated, when two users are connected to the SCIAM Collaborative Code Editor, there is an average latency of 6ms. However, when a third user connects to the collaborative code editor, the average latency is reduced to approximately 3ms, suggesting an efficient message broadcast mechanism. When a fourth user joins the session, the latency is slightly increased to 4 ms, but this latency indicates high stability of the synchronization process in the sub-10 ms range.The above data indicate the efficiency of the real-time communication architecture based on Socket.IO, together with the backend server based on Flask-threaded, for handling the collaborative editing process in real-time.

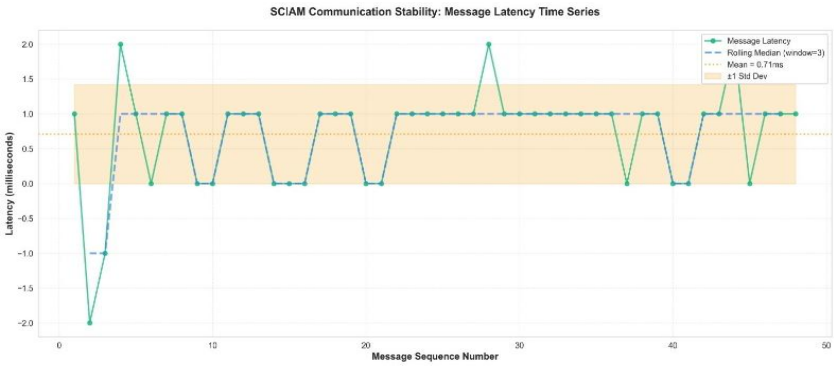


Figure 6.2: Graph of Communication Stability

Figure 6.2 depicts the Communication Stability: Message Latency Time Series of the SCIAM software, emphasizing its stability as well as responsiveness of its communication channel. Here, the x-axis corresponds to the number of the message, while the y-axis displays the latency in milliseconds. From the results, one can see that the majority of latencies are low and are consistently grouped at the mean latency value of about 0.7 milliseconds. This illustrates the efficiency of message sending among the participants. Moreover, the presence of the rolling median confirms the stability of the system because the difference from the mean is minimal despite the increasing number of messages. Some insignificant peaks may appear on the time series; however, they are located within the standard deviation level. It means that no system instability occurs. Such a result ensures good collaboration when developers should use real-time communication. Thus, the study proves that SCIAM's WebSocket-based communication infrastructure operates efficiently and provides effective simultaneous messaging.

Error Type	Cases	Top-1	Top-2	Acc-1	Acc-2
ZeroDivisionError	4	4	4	100%	100%
IndexError	4	4	4	100%	100%
KeyError	4	4	4	100%	100%
TypeError	4	1	1	25%	25%
ValueError	4	0	2	0%	50%
OVERALL	20	13	15	40.6%	46.9%

Table 6.1: Performance of AI Assistance Model

The result table shows an organized way of analyzing the performance of the proposed AI Assistance model when addressing runtime errors in the SCIAM system, including identifying run-time problems in programs and making explainable suggestions when programs are

running. In this case, it can be observed that the AI Assistance model performs really well, obtaining perfect performance of 100% with respect to Top-1 and Top-2 accuracy for the most frequent errors, including ZeroDivisionError, IndexError, and KeyError, while syntactical errors and naming errors were addressed through the editor to guarantee correct responsibility separation. Additionally, the AI Assistance model demonstrates good behavior when addressing TypeError since it depends on the context and the code written in the program with only 25% of accuracy. On the other hand, there is some variability when addressing ValueError, in which the model shows 0% accuracy for Top-1 but reaches 50% accuracy for Top-2 because useful suggestions could be made for this error. As can be seen from the previous information, the proposed SCIAM system shows high performance when addressing runtime errors, obtaining 65% Top-1 and 75% Top-2 accuracy with respect to 20 test cases.

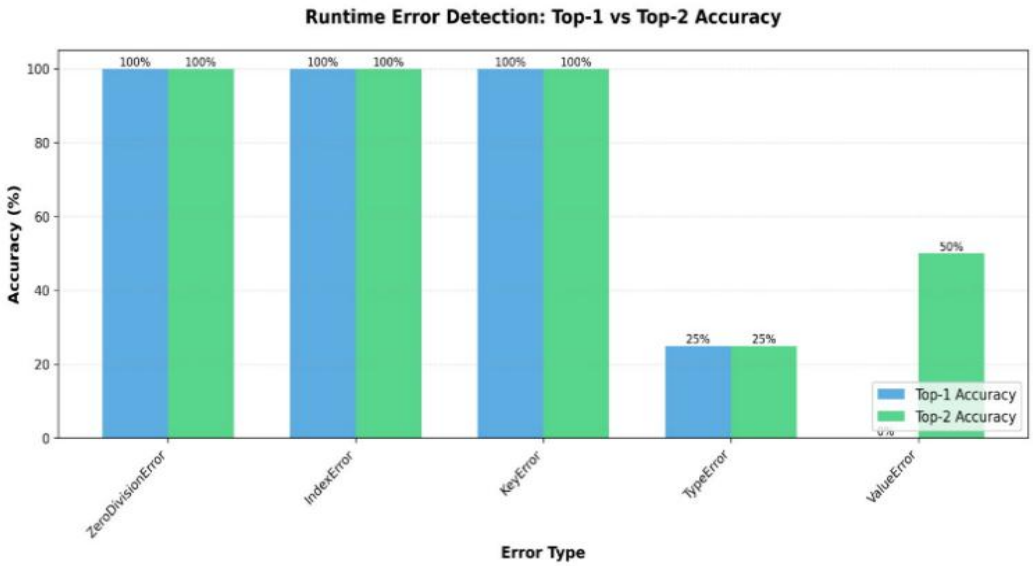


Figure 6.3 : Error Detection

The graph examines the accuracy of the AI Assistance model on various types of runtime errors based on Top-1 and Top-2 predictions. Figure 6.3 Error Detection The model performs outstandingly well on general runtime errors such as ZeroDivisionError, IndexError, and KeyError, with 100% accuracy, demonstrating its high level of dependability on frequent runtime errors. However, the model's performance drops on context-specific errors such as TypeError, with only 25% accuracy, due to the complexity of type analysis. The Top-1 accuracy on ValueError is 0%, but Top-2 accuracy increases to 50%, suggesting good alternative predictions.

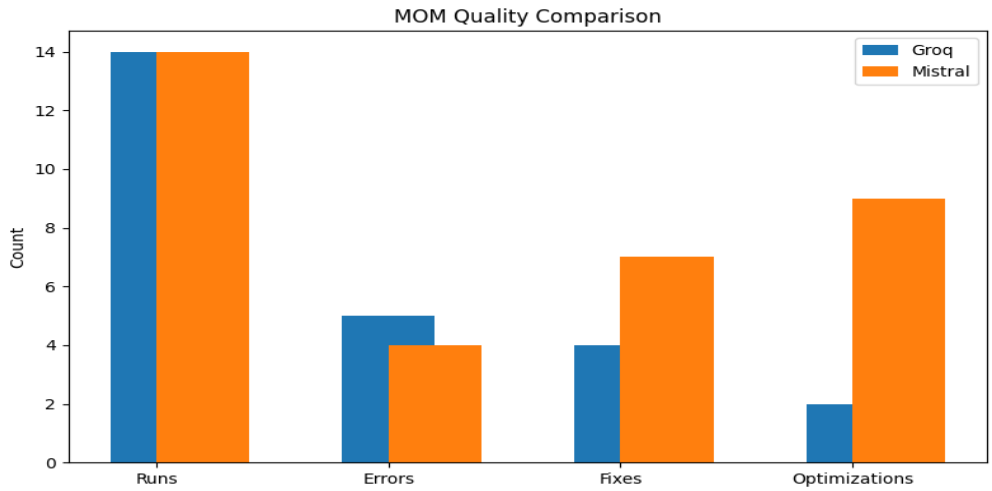


Figure 6.4 : MOM Quality Comparison

Figure 6.4 compares both graphically and using console results the Groq and Mistral models' ability to detect key elements of collaborative development discussions in creating Minutes of Meeting (MOM) in the SCIAM project. According to the table below, both models equally well identify executions with 14 occurrences. This is a demonstration of the models' awareness of the discussion flow and workflow structure. Regarding the detection of errors, Groq does slightly better than Mistral, as it recognizes 5 points related to errors, compared to Mistral's 4. The same cannot be said about fix recognition and optimization discussions, as the second model outperforms the first by a wide margin. Mistral is evidently able to comprehend optimization and fixes better, even if the speakers do not use exact terminology. Finally, both models can be considered equally proficient at recognizing other MOM-specific elements, such as a balanced number of words, the inclusion of timelines, and accurate naming of participants. Thus, they satisfy the criteria of an effective MOM system (structure, clarity, and accountability). Nevertheless, Mistral receives more points because it detects more insights from conversations, such as solutions to problems and optimizations. In this regard, it works much better than Groq.

Chapter 7

Conclusion

The project involved the creation of the SCIAM (Smart Collaborative IDE with AI Support and Session Management). This software addresses the major challenges that software development poses nowadays. First, it combines code editing, AI-assisted code writing, and automatic session management. These elements make the system more productive as they reduce routine actions, prevent mistakes, and suggest efficient code solutions. Besides, the real-time collaboration functions improve teamwork as they provide greater visibility and accountability. In addition, the ability to prevent losing the results due to the lack of automatic session management is a valuable feature of the software. Another advantage is the integration of tools for performance monitoring and optimization. Additionally, SCIAM can be applied in educational purposes as it provides AI-assisted feedback.

With customizable plug-ins and support for API, the system becomes flexible for different projects. Through the integration of communication, debugging, and versioning, SCIAM minimizes reliance on a number of separate tools for developers. With its user-friendly interface and efficient workflows, SCIAM improves the developer experience and speeds up the process of familiarizing oneself with the toolset. Finally, SCIAM contributes to constant growth and innovation by offering feedback and recommendations throughout the development process. Thus, overall, the project shows how collaboration and AI applications contribute to changing the way we work in software engineering. SCIAM sets a basis for creating an innovative IDE in the future. Future improvements could involve multi-modal AI features, DevOps integration, and AR/VR collaboration.

Chapter 8

Future Scope

- Sophisticated AI Implementation – Intelligent algorithms used for auto-completion, debugging, optimizing code, and even translation from natural language to code.

- Mobile & Web Optimization – Expand the application to work across smartphones, tablets, and web applications as well.

- Scalable Cloud Capabilities – Improve cloud capabilities to cater to corporate customers, supporting multiple thousand simultaneous developers.

- Improve Security & Privacy – Blockchain version control, zero-trust security model, and encrypted collaboration sessions between developers.

- Custom Plug-in Architecture – Enable developers to create and install plugins, extensions, and automate development workflows.

- Corporate DevOps Integration – Provide end-to-end support for continuous integration and deployment pipelines, including container orchestration.

- Smart Resource Management – Intelligent allocation and scheduling of work and tasks for efficient use of developer skills.

- Global Community for Developers – Create a community where people around the world can collaborate with one another.

- Polyglot Language Support – Support for multiple languages, frameworks, and even emerging technology stacks to increase reach.

- Code Quality Analysis Engine – Advanced analytics for code quality, performance bottlenecks, and risk assessment.

Bibliography

[1] Htoo Htoo Sandi Kyaw, Keiichi Kaneko, Nobuo Funabiki,A Proposal of the Gamification of Programming Learning Assistant System,2024

[2] Vaishali Savale, Vedashri Chaudhari, Tejaswini Wanare, Balamurali Nambiar, Manthan Vasant, Sanskruti Umak, CodeQuest - A Comprehensive Code Learning and Collaboration Platform,2024

[3] Rijul Saini, Gunter Mussbacher,Towards Conflict-Free Collaborative Modelling Using VS Code Extensions,2021

[4] Mingjie Wang, Bicheng Fang, Jinfeng Jiang, Hongfei Fan, Annotation-based Semantic Conflict Prevention in Real Time Collaborative Programming: Approach, Techniques, Prototype, and User Study,2024

[5] Khushwant Virdi, Anup Lal Yadav, Azhar Ashraf Gadoo, Navjot Singh Talwandi,Collaborative Code Editors - Enabling Real-Time Multi User Coding and Knowledge Sharing,2023

[6] Jinfeng Jiang, Yuxiang Xie, Bicheng Fang, Mingjie Wang, Hongfei Fan, Building Temporary Isolated Workspace in Real-Time Collaborative Programming Environment,2023

[7] Mihir Agrawal, Jatin Goyal, Mradul Goyal, Pratham Sukhija, Jaya Sharma, D. Franklin Vinod, CodeXchange: Leaping into the Future of AI-Powered Code Editing,2024

[8] Ankita Amburle, Cheryl Almeida, Oswin Lopes, Nathan Lopes, AI Based Code Error Explainer using Gemini Mode,2024

[9] Lingchen Huang, Huazi Zhang, Rong Li, Yiqun Ge, Jun Wang, AI Coding: Learning to Construct Error Correction Codes,2020

[10] Matteo Ciniselli, Nathan Cooper, Luca Pascarella, Antonio Mastropaolo, Emad Aghajani, Denys Poshyvanyk, Massimiliano Di Penta, Gabriele Bavota, An Empirical Study on the Usage of Transformer Models for Code Completion,2021

[11] Zixiao Zhao, Jing Sun, Cheng-Hao Cai, Zhiyuan Wei, Code Generation Using Self interactive Assistant,2024

[12] Haibin Lai, Peng Peng, Yuhui Lv, Peiwei Cai, Qiongxiong Ma, Zhun Zhang, The Development of a Collaborative Learning Tool for AI Education: Corobolab,2025

[13] Saptaparni Kumar, Haochen Pan, Roger Wang, Lewis Tseng,LiteDoc: Make Collaborative Editing Fast, Scalable, and Robust,2020

Appendix

I. Tools and Technologies Used

- **Programming Language:** Python
- **Libraries and Frameworks:**
  - Flask – Backend web application framework for routing and server-side logic.
  - Flask-SocketIO – Real-time bidirectional communication for collaborative coding using WebSockets.
  - MySQL Connector – Database connection and session/user data management.
  - PyTorch – Local AI model loading and inference for intelligent debugging guidance.
  - Ollama (DeepSeek-Coder) – Local LLM-based code analysis and logic review.
  - Requests – API communication for external AI services.

- **Development Tools:**
  - Visual Studio Code – Code development and debugging.
  - Google Colab – AI model training and experimentation.
  - MySQL Workbench – Database design and management.

II. Datasets Used

- Python Error Guidance Dataset - Used to train the AI-based debugging guidance model for detecting and analyzing runtime and syntax errors.
- Custom Code Error Examples Dataset – Includes structured examples of Python exceptions such as ZeroDivisionError, TypeError, IndexError, and InputError for model guidance and rule-based fallback.
- Code Execution Trace Data – Used for capturing stdout, stderr, latency logs, and execution history for analysis and reporting.

III.Hardware and Software Configuration

- **Processor:** Intel Core i5 (10th Gen or above)
- **RAM:** 8 GB
- **GPU:** NVIDIA GPU (optional for AI model training)
- **Operating System:** Windows 10/11
- **Software:** Python 3.x, Jupyter Notebook, VS Code,MySQL,Ollama (for local LLM inference)

IV. Code Snippets Overview

- **Real-Time Collaborative Editor -** Python implementation using Flask-SocketIO for multi-user collaborative code editing with role-based access control (Host, Writer, Viewer).
- **Code Execution Engine -** Implementation of secure Python code execution using subprocess with support for interactive input, timeout control, and optional Docker sandboxing.
- **AI Debugging Guidance -** Integration of local transformer model and rule-based fallback system for automatic error explanation and debugging suggestions.
- **Invite and Collaboration Control System** - Real-time invitation, participant tracking, writer handover, and collaboration event logging.
- **Latency and Execution Monitoring -** System for tracking code synchronization latency, network round-trip time, execution logs, and performance metrics.

V. Copyright

Copyright Office
Government of India

Acknowledgement Slip
(Date: 18/02/2026)

Diary Number: **SW-7467/2026-CO**	Form Received: **Online**
Copyright Reg. of: **Computer Software**	Titled: **SCIAM**

Communication Address

Name	Address	Phone Number
RISHI BIJE	1204/B-WING, MULUND AMBIKA NAGAR CHS,J.N ROAD , MULUND WEST, MUMBAI-400080	08108106845

Financial Details

Payment ID	Amount	Bank Name	Payment Mode	Payment Date
450203	500			18/02/2026

* For future communication please mention this DIARY No.

INSTRUCTIONS

For the purpose of processing the application, following documents are mandatory to send by post along with the acknowledgement slip(Office Copy).

1.	2 Copies of work
2.	DD/IPO of Rs.(as applicable) per work favouring Registrar Copyright Office payable at New Delhi (Not applicable for online payment)
3.	Authorization from author/publisher
4.	If the work is being used on goods or capable of being used on the goods
5.	If the application is being field through attorney, a specific power of attorney in original duly signed by the applicant and accepted by the attorney
6.	Search Certificate from Trade Mark Office(TM-60) (*Only in case of Artistic work*).
7.	Applicant must take a print out of the application, sign it and send along with the other documents.

Kindly send the above documents within 30 Days from the date of online submission on the following address given by herewith: .

Office of the Registrar of Copyrights
Copyright Office, Department for Promotion of Industry & Internal Trade
Ministry of Commerce and Industry
Boudhik Sampada Bhawan,
Plot No. 32, Sector 14, Dwarka,
New Delhi-110078
Email Address: copyright@nic.in
Telephone No.: 011-28032496

APPLICANT'S COPY
THANK YOU

Copyright Office
Government of India

Acknowledgement Slip
(Date:18/02/2026)

Diary Number: **SW-7467/2026-CO**	Form Received: **Online**
Copyright Reg. of: **Computer Software**	Titled: **SCIAM**

Communication Address

Name	Address	Phone Number
RISHI BIJE	1204/B-WING, MULUND AMBIKA NAGAR CHS,J.N ROAD , MULUND WEST, MUMBAI-400080	08108106845

Financial Details

Payment ID	Amount	Bank Name	Payment Mode	Payment Date

450203	500			18/02/2026

* For future communication please mention this DIARY No.

INSTRUCTIONS	
For the purpose of processing the application, following documents are mandatory to send by post along with this acknowledgement slip(Applicant's Copy).	
1.	2 Copies of work
2.	DD/IPO of Rs.(as applicable) per work favouring Registrar Copyright Office payable at New Delhi (Not applicable for online payment)
3.	Authorization from author/publisher
4.	If the work is being used on goods or capable of being used on the goods
5.	If the application is being field through attorney, a specific power of attorney in original duly signed by the applicant and accepted by the attorney
6.	Search Certificate from Trade Mark Office(TM-60) (*Only in case of Artistic work*).
7.	Applicant must take a print out of the application, sign it and send along with the other documents.
Kindly send the above documents within 30 Days from the date of online submission on the following address given by herewith: .	
Office of the Registrar of Copyrights Copyright Office, Department for Promotion of Industry & Internal Trade Ministry of Commerce and Industry Boudhik Sampada Bhawan, Plot No. 32, Sector 14, Dwarka, New Delhi-110078 Email Address: copyright@nic.in Telephone No.: 011-28032496	

OFFICE COPY
THANK YOU

A. File Structure Directory

1. **Frontend Directory Structure**

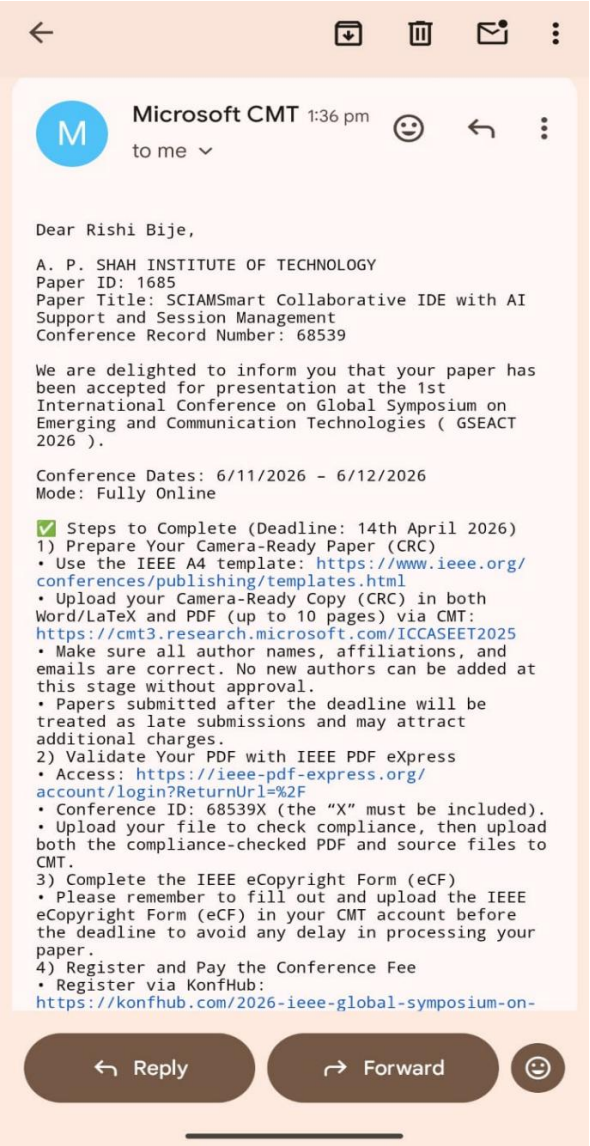
```
|-- templates
|   |-- register.html
|   |-- login.html
|   |-- homes.html
|   |-- editors.html
|   |-- session_reportn.html
```

2. **Backend Directory Structure**

```
|-- appnew.py

|-- train_codet5.py
```

VI. Research Paper



Microsoft CMT 1:36 pm
to me

Dear Rishi Bije,

A. P. SHAH INSTITUTE OF TECHNOLOGY
Paper ID: 1685
Paper Title: SCIAMSmart Collaborative IDE with AI Support and Session Management
Conference Record Number: 68539

We are delighted to inform you that your paper has been accepted for presentation at the 1st International Conference on Global Symposium on Emerging and Communication Technologies (GSEACT 2026).

Conference Dates: 6/11/2026 – 6/12/2026
Mode: Fully Online

✅ Steps to Complete (Deadline: 14th April 2026)
1) Prepare Your Camera-Ready Paper (CRC)
• Use the IEEE A4 template: https://www.ieee.org/conferences/publishing/templates.html
• Upload your Camera-Ready Copy (CRC) in both Word/LaTeX and PDF (up to 10 pages) via CMT: https://cmt3.research.microsoft.com/ICCASEET2025
• Make sure all author names, affiliations, and emails are correct. No new authors can be added at this stage without approval.
• Papers submitted after the deadline will be treated as late submissions and may attract additional charges.
2) Validate Your PDF with IEEE PDF eXpress
• Access: https://ieee-pdf-express.org/account/login?ReturnUrl=%2F
• Conference ID: 68539X (the "X" must be included).
• Upload your file to check compliance, then upload both the compliance-checked PDF and source files to CMT.
3) Complete the IEEE eCopyright Form (eCF)
• Please remember to fill out and upload the IEEE eCopyright Form (eCF) in your CMT account before the deadline to avoid any delay in processing your paper.
4) Register and Pay the Conference Fee
• Register via KonfHub:
https://konfhub.com/2026-ieee-global-symposium-on-

Reply Forward

Figure 8.1 : Research Paper Acceptance Mail

The above figure shows the official acceptance notification received via Microsoft CMT for the research paper titled "SCIAM - Smart Collaborative IDE with AI Support and Session Management". The paper has been accepted for presentation at the 1st International Conference on Global Symposium on Emerging and Communication Technologies held in Hyderabad, India, in 2026. The acceptance confirms that the work has been reviewed and approved by experts in the field of Artificial Intelligence and Machine Learning.

SCIAM—Smart Collaborative IDE with AI Support and Session Management

Sarala Mary
Department of CSE (Data Science)
A.P. Shah Institute of Technology
Thane (M.H), India 400615
saralas@apsit.edu.in

Rishi Bije
Department of CSE (Data Science)
A.P. Shah Institute of Technology
Thane (M.H), India 400615
rishibije2@apsit.edu.in

Dhanraj Bacche
Department of CSE (Data Science)
A.P. Shah Institute of Technology
Thane (M.H), India 400615
dhanrajbacche53@apsit.edu.in

Yash Kamble
Department of CSE (Data Science)
A.P. Shah Institute of Technology
Thane (M.H), India 400615
yashkamble52@apsit.edu.in

Shuban Devanpelli
Department of CSE (Data Science)
A.P. Shah Institute of Technology
Thane (M.H), India 400615
shubandevanpelli21@apsit.edu.in

Richa Singh
Department of CSE (Data Science)
A.P. Shah Institute of Technology
Thane (M.H), India 400615
richasingh@apsit.edu.in

***Abstract*— SCIAM is an interactive development environment where the goal is to make the creation of software easier, smarter, and more effective. SCIAM is the integration of an actual code-writing environment, an intelligent code-writing environment powered by AI, and an intelligent session management environment. SCIAM provide effective error identification, and expert code suggestions, hence allowing the programmer to develop high-quality code in no time and without tedious work. The key benefit of SCIAM is its ability to handle code sessions in an extremely intelligent manner. SCIAM is capable of automatically tracking, saving, and reloading code working sessions on the collaborative coding environment of SCIAM, ensuring there is no lost data and version issues. SCIAM will integrate code-writing environments and code-debugging environments to make it easier to code and decrease dependencies on various environments. SCIAM will encompass security and performance analytics in its design. With a major emphasis on identity and access management, only those individuals can get access to the sessions as well as the resources who are permitted to do so. This will result in the creation of an ideal environment for development. However, the major drawback is the emphasis on analytical tools as well. This will assist the teams in getting an understanding of their performance as well. This will assist the teams in making appropriate decisions as well. This will result in the creation of an ideal focus on the environment of development as well. Except for the development teams, SCIAM is an ideal environment for educational institutions as well. There will be efficiency in the environment because it is cloud-based. There will be an ideal focus on the environment of development because of the windows' cross-platform compatibility. SCIAM is an ideal representation of the next generation of the cooperative environment where the domains of education, industry, and research are interlinked.**

I. Introduction

Generally, the overall process of software development is observed to be problematic regarding ineffective cooperation, decreased productivity, and inconsistent code quality, which might lead to delaying the projects and increasing the possibility of errors. Most traditional development is based on a variety of self-contained tools, relying on manual session tracking and muddled communication practices and might lead to time wastage, version conflicts, and delaying critical decision-making. The Smart Collaborative IDE with AI Support and Session Management (SCIAM) is developed such that it is free from all these aforementioned issues and provides a single intelligent and effective development environment.

SCIAM provides a single platform that is a combination of code editing, support systems, and management systems. Whatever changes are done in the code and contributions made by members are accurately recorded, and it ensures clarity and accountability throughout the entire development process. Code writing is made smarter by features such as intelligent code suggestions, error detection, and guidance that enable programmers to write cleaner code with much less effort. Recording sessions is automated so that no progress is lost and prevents those kinds of conflicts which come into play during collaboration.Moreover, the platform facilitates real-time collaboration and integrates in-built communication tools; this allows every member of the team to be updated and connected with happenings always. Such clear visibility into project progress helps organizations make quicker and better decisions.

AI-driven feedback helps students and professionals improve their coding skills in an educative and training environment. With its intuitive interface, secure collaboration, and complete activity tracking, SCIAM is a next-generation development ecosystem, designed to empower teams towards building better software faster and more productively.

II. RELATED WORK

The following literature review discusses some of the existing solutions for collaborative coding and AI-assisted development, such as cloud-based IDEs, smart debugging, and version control systems. It shows that unresolved issues include multi-user simultaneous editing, avoidance of version conflicts, maintenance of continuous sessions, and seamless integration of communication and development tools, especially in teams involving a large number of participants.

Kyaw et al.[1] have proposed a gamification-assisted learning tool in programming to enhance the interest of new learners in programming.

Savale et al. [2] proposed CodeQuest, an online collaborative space for learning data structures in a real-time coding environment with an integrated AI chatbot. However, CodeQuest lacks the following factors: personalized learning paths for learners, support for more programming languages, and embedded gamification aspects.

Saini and Mussbacher [3] introduced tColab, an extension of the Visual Studio Code (VS Code) editor that facilitates conflict-free collaborative modeling through CRDTs. Although it handles concurrency quite effectively, it only works with the VS Code editor and does not resolve conflict semantics semantically.

Wang et al. [4] proposed an annotation-based approach to avoid semantic conflicts during collaborative coding using an IntelliJ plugin. While it is beneficial for maintaining consistency in code, it embeds manual overhead and is limited to only one IDE.

Virdi et al.[5] analyzed real-time collaborative code editor solutions employing WebSockets and Operational Transform. The proposed solution performs well for a smaller team but encounters some latency, synchronization, and scalability issues.

Jiang et al.[6] have outlined the RECON approach that facilitates isolated debugging and compilation in collaborative coding. Although it addresses the reliability issues, it also consumes more resources and restricts editing freedom in isolation.

Huang et al.[7] proposed an AI algorithm for building error-correcting codes with the use of reinforcement learning. Although the algorithm performed well, it is computationally expensive and restricted to simulation environments.

Amburle et al.[8] have created an AI assistant that uses the Gemini model to explain code errors in real time. It is quite useful for debugging purposes but lacks support for many programming languages and requires more detailed explanations.

Omi et al.[9] proposed a deep learning approach for finding several programmers from code based on AST properties. Although it is highly accurate, it is Java-centric and scaleable issues exist.

Lai et al. [10] introduced Corobolab, which is a collaborative AI learning platform that aims to serve students from kindergarten to high school with real-time programming assistance. However, it needs improvement in terms of user interface, network, and adaptability for advanced learners.

Agrawal et al. [11] proposed an AI-powered collaborative code editor called CodeXchange, which has neural translation, GPT-3 assistance, and real-time editing. Still, this faces challenges in terms of computational overhead, complexity in conflict resolution, security risks, and scaling.

Ciniselli et al. [12] proposed an empirical study on the use of Transformer models such as RoBERTa and T5 for code completion. They observed better results on smaller code blocks but with high computational complexity, accuracy reduction on code blocks, and language-dependent results.

Zhao et al. [13] put forward a self-interactive coding model called SelfCoder, which uses GPT-4. However, the model still had challenges of reliability, ambiguity, costs, and lack of domain knowledge.

Kumar et al. [14] proposed a scalable collaborative editor named LiteDoc with section-based single-writer semantics to provide correctness with a trade-off of less concurrency, features, and usability compared to fully flexible collaborative editors. The proposed model mainly emphasizes openness.

Junbang Ma. [15] proposed a model for web application testing and optimization using AI and deep learning and reinforcement learning techniques, which improves testing speed and code quality but is computationally expensive and less adaptable and language-friendly.

III. PROPOSED SYSTEM ARCHITECTURE

The following is a diagram of the overall architecture of the SCIAM platform that demonstrates how all the different parts of it work together for collaborative software development. The different modules of it include the collaborative IDE environment, AI code assistant, management of sessions, real-time communication, and report generation that all work well together for the efficient collaboration of software development while keeping access secure.

The architecture also describes how more than one user is working on code simultaneously without any conflicts and loss of data. The AI assistant is also monitoring the code modifications for useful suggestions and debugging purposes. Session management assists in managing sessions regarding code versions and logs for any activity performed and also recovers any lost code modifications performed previously. The architecture is designed to enhance productivity and scalability.

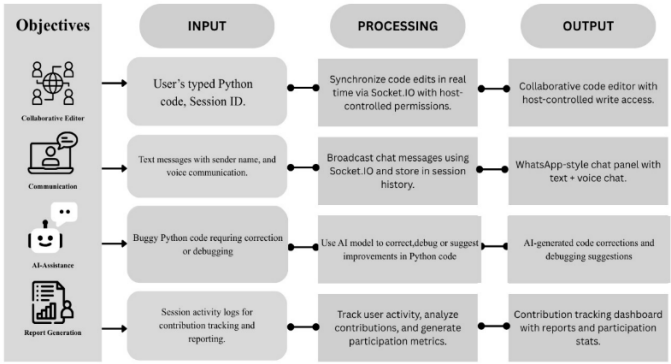


Fig. 1. Proposed System Architecture

A. Collaborative Editor Module

Collaborative Editor Module: This is the central part of the SCIAM platform and aims to make it possible for several users to co-edit a common code space. SCIAM uses a Flask server with Socket.IO for real-time communication between the client and server with low latency. With every modification of code, transition of control, or change of a file by a user, the

modification is broadcast to the server and sent to all participants in a collaborative session. This means all participants see the latest version of all code under modification. SCIAM relies upon a session state handled centrally on its server to synchronize its participants.

The editor handles live code sharing, role management as a writer, file management, and file execution-triggered updates. The solution records critical session activities like code changes, write privileges, and execution activities, which are used later to generate the reports. This architecture gives an ideal environment to work or to teach in, as it is stable, interactive, and controlled.

Therefore, the Collaborative Editor Module provides a trustworthy common coding area where multiple users can code and develop programs collaboratively using coordinated views among all the connected clients.

B. Communication Module

This Communication Module supports real-time communication for all participants directly within the SCIAM environment. The Communication Module uses socket communication, allowing all participants to communicate through instant chat messages while they are working within their coding workspace.

All chat messages will be linked to the current session and will be in sync with all participants. In this manner, users can discuss errors, discuss logic, assign tasks, and help lead each other in their process without exiting the system. Messages are stored in session memory and can be included in the session report for traceable and context-aware discussion.

By embedding communication within the collaborative editor, SCIAM reduces reliance on external tools, improves coordination, and ensures that discussions remain tightly connected to the coding activity

C. AI Assistance Module

The AI Assistance Module is an explainable guidance module in SCIAM, which is intended to assist the developer in code execution and debug processes instead of code corrections. Unlike other code-correcting approaches, the AI Assistance Module does not attempt to correct the user's code but instead identifies the code execution process and generates structured help advice on the code execution error. Such assistance is mainly targeted at code execution and data access-related errors like arithmetic or invalid-container-related errors in code execution. Syntax-related or name resolution-related issues are not intended to be handled in this module.

Guidance is provided in a non-intrusive fashion in terms of summaries, explanations of causes, and solution suggestions for developers to retain control over their modifications in their code. Users are given a choice to apply recommended solutions or simply ignore them to continue their activities uninterrupted in their collaborative session. Giving emphasis to explanation instead of making automatic repairs helps in learning-oriented debugging.It is particularly helpful for novice and Collegial programmers because it will remove the burden of error interpretation and make it easier to understand code and correctly teach programmers how to do it correctly. It gives cooperative and clear definitions of erroneous programs instead of an automated program that corrects it.

1. Core Model Formulation

$$\hat{e} = f(C, T)$$

Where :

- C = code context
- T = runtime traceback
- $f(\cdot)$ = trained AI guidance model
- $\hat{e}$ = predicted error type

2. TOP-1 and TOP-2 Formulation

Let :

- $E = e_1, e_2 \ldots, e_k$ be the top-k predicted error types
- e^* be the ground truth error

Then:

Top -1 Accuracy:

$$AccTop_1 = \frac{1}{N}\sum_{i=1}^{N} 1\left(e_{1=e^*}^{(i)}(i)\right)$$

Top-2 Accuracy:

$$Acc_{Top_2} = \frac{1}{N}\sum_{i=1}^{N} 1\left(e^{*(i)} \in \left\{e_1^{(i)}, e_2^{(i)}\right\}\right)$$

3. Precision , Recall , F1 (Main Evaluation Equations)

For each error type e :

$$Precision_e = \frac{TP_e}{TP_e + FP_e}$$

$$Recall_e = \frac{TP_e}{TP_e + FN_e}$$

$$F1_e = \frac{2 \cdot Precision_e \cdot Recall_e}{Precision_e + Recall_e}$$

Where :

- TP = correctly identified errors
- FP = wrongly predicted as that error
- FN = missed detections

D. Report Generation Module

Report Generation Module: This module is a significant analytical part of SCIAM. During every session, it maintains organized data regarding participants, update in code, execution trail, chatting action, session time, as well as code snapshots for every user.

After a session is complete, this information is compiled into a session report. The session report contains the following:

1. A session description (starting time, ending time, length, participants)

2. Contribution summaries by user.

3. Code execution history with errors and results

4. Graphs illustrating user activities and trends of participation.

An example Minutes of Meeting (MoM) generated by AI, describing the flow, fixes, and outcome of the meeting

These reports are kept in a database using MySQL and are protected by access if users are not logged in. The reporting system equally allows the hosts to look back upon past sessions and reflect on collaboration patterns and monitor the progress of the project. Such a system is most appropriate in an academic setting or in development environments where productivity can be monitored. SCIAM goes beyond a coding aid and becomes a collaborative intelligence environment.

1. Session Duration Calculation

$$T_{session} = T_{end} - T_{start}$$

Where :

- $T_{session}$ = Total session duration
- T_{start} = Session start timestamp
- T_{end} = Session end timestamp

2. User Contribution Score

$$C_u = \alpha E_u + \beta R_u + \gamma Mu$$

Where:

- C_u = Contribution score of user u
- E_u = Number of code edits by user
- R_u = Number of successful code executions
- Mu = Number of messages / interactions
- α, β, γ = weighting factors

IV. RESULT AND ANALYSIS

This paper aims to assess the effectiveness of SCIAM (Smart Collaborative IDE with AI Support and Session Management), as tested in real-world, scenario-based experiments. This will help in understanding the effectiveness of SCIAM for real-time collaboration, avoiding code conflict, accelerating coding, and efficient AI assistance in collaborative coding sessions. For assessing SCIAM, comparisons are conducted between SCIAM and traditional single-user collaborative code software, as well as basic collaborative software that does not include AI assistance or intelligent session management features. The testing reveals that the collaborative, intelligent, and automated session management capabilities in SCIAM are more effective in real-world coding sessions.

A. Performance of Real Time Synchronization

Figure 2 represents how properly the SCIAM Collaborative Code Editor is able to produce real-time synchronization between the team of programmers during live coding sessions. It is clear from the above diagram that, where there are two programmers, the latency is around 6ms, but where there are three programmers, it surprisingly decreases below 3ms, indicating optimal handling and processing of events and message broadcasting, and finally, where there are four programmers, it again slightly increases to 4ms, still within the optimal range.

Overall, the trend in the findings appears to emphasize that SCIAM has the capability to achieve a low latency value that is less than 10 ms, ensuring a collaborative coding atmosphere that runs smoothly

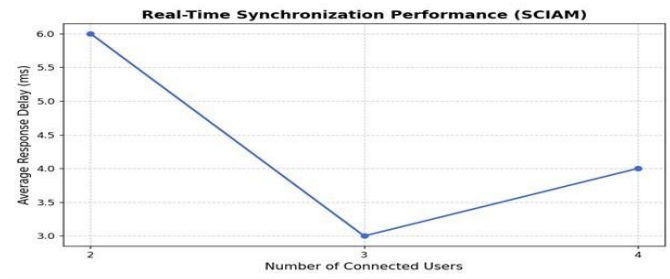


Fig. 2. Performance of Real Time Synchronization

Using Socket.IO in the collaborative communication platform has the ability to address the issue of multiple edits in a short span of time. The variation in the latency values, regardless of the user loads, appears to emphasize the efficiency in the synchronization accuracy. The caching system in the local setup appears to reduce redundancy in the transmission of data. The findings clearly emphasize that the designing of the architecture has efficiency in a collaborative coding setup.

B. Graph of Communication Stability

In Figure 3, the Communication Stability: Message Latency Time Series for the SCIAM platform is shown. This figure not only depicts the stability of the communication system in SCIAM but also indicates how responsive the real-time communication module is to any latency in messages. In these figures, the x-coordinate corresponds to the sequence number of the messages exchanged in the course of collaboration scenarios, and the y-coordinate corresponds to the latency times of the messages in milliseconds. It can be concluded that most latency times are well within the minimum range and are grouped closely around the mean latency time of approximately 0.7 ms as indicated by the dotted mean line in the figures. This indicates that the time taken for messages to be transmitted between collaborators is minimal.

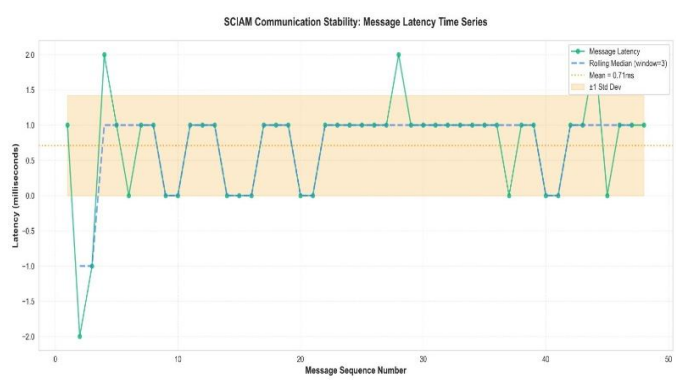


Fig. 3. Graph of Communication Stability

The rolling median plot again supports the stability of the system, with very small variations despite the increase in the number of messages. Some minor peaks are visible, but they are all within the standard deviation zone, showing that the system does not malfunction. The graph marked with the one standard deviation zone shows that the latency is always within predictable limits. Responsiveness is always essential in collaborative programming, where discussions, explanations, and coordination are instantaneous.

These results not only justify SCIAM's efficient communication system through WebSockets, which handles concurrent communication of message broadcasting successfully without any degradation of performance, but they also ensure efficient communication performance in SCIAM even in constant communication among various users. Based on this graphical result, this result clearly shows how SCIAM can guarantee a communication experience like real-world conversations, which allows developers to work together successfully in distributed soft development environments.

C. Performance of AI Assistance Model

The result table provides a well-organized and easy-to-understand assessment of the performance of the AI Assistance model on runtime error detection and explanation, thus further justifying its purpose in the SCIAM system. The AI model is mainly designed to identify patterns at the execution time level and offer explainable suggestions to the user at the time of program execution, whereas syntax- and name-related errors are specifically left to the editor or IDE to maintain a proper division.

The result table provides very useful information on the effectiveness of the AI model in responding to various categories of runtime errors and also points out the specific situations in which the AI model's suggestions are most effective. For general execution errors such as ZeroDivisionError, IndexError, and KeyError, the model performs outstandingly well, with 100% accuracy in both Top-1 and Top-2 predictions for all test cases. This clearly indicates that the AI model is extremely reliable when dealing with general runtime errors that programmers usually face during the testing and execution stages. However, for TypeError, the model performs moderately well, with 25% accuracy, due to the contextual nature of type-related errors, which are highly dependent on variable usage and program logic.

The most prominent difference is seen in the case of ValueError, where Top-1 accuracy is 0% but increases to 50% for Top-2 accuracy, which clearly indicates that although the most appropriate solution may not always be the first recommendation, the system is still able to provide helpful information from its recommended list. On the whole, the runtime analysis test among 20 test cases gives a Top-1 accuracy of 65% and Top-2 accuracy of 75%, which clearly reiterates the excellent performance of the system in the context of execution-related errors. The presence of zero accuracy in some cases is also in line with the SCIAM philosophy of providing editor-level support for syntax and naming errors, which leaves the AI Assistance model solely responsible for runtime debugging and execution-related problem-solving, thereby complementing the conventional development environment rather than replacing it.

The graph examines the accuracy of the AI Assistance model on various types of runtime errors based on Top-1 and Top-2 predictions.

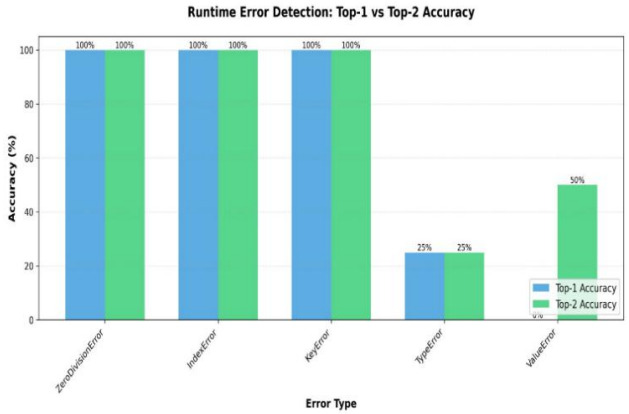


Fig. 4. Error Detection

The model performs outstandingly well on general runtime errors such as ZeroDivisionError, IndexError, and KeyError, with 100% accuracy, demonstrating its high level of dependability on frequent runtime errors. However, the model's performance drops on context-specific errors such as TypeError, with only 25% accuracy, due to the complexity of type analysis. The Top-1 accuracy on ValueError is 0%, but Top-2 accuracy increases to 50%, suggesting good alternative predictions.

D. MOM Quality Comparison

Figure 5 illustrates the graphical representation along with the console results that bring out an effective comparison of the ability of both Groq & Mistral models in generating MOM in the SCIAM project. The discussion of this figure aims at examining the models' ability to identify related aspects of cooperative development discussion, such as execution runs, errors, corrections, & optimisation attempts. Both models have an equal capability to recognize execution runs since they both correctly detect 14 of them, pointing out they are able to correctly interpret the entire process & structure of an entire development dialogue. Concerning error detection capability, Groq performs slightly better since it spots five points related to errors in comparison to Mistral, which spots four, pointing out it has an edge in error detection. Mistral has an evident strong capability of recognizing content that is related to solutions.

This strength is further observed in the optimization area, where the number of points related to optimization observed by Mistral is notably higher, indicating its capacity to understand general discussions about optimizing performance and efficiency, even in an indirect form. Other similarity metrics, including similarity in the number of words, use of timelines, and proper labeling of participants, indicate that both systems fare well in fulfilling the criteria of a good MOM system, including understandability, organization, and accountability.

Fig. 5. MOM Quality Comparison

However, despite these points, the result analysis reveals a marked difference in the scores, where the higher score goes to Mistral because of its marked excellence in pinpointing fixes and optimization points, which are more important for a meaningful meeting MOM. In conclusion, although Groq is reliable regarding pointing out workflow actions and flaws, a more insightful, action-loaded, and thoughtful MOM system is provided by Mistral, which is more suitable for SCIAM's AI-driven reporting in a co-working environment for software development.

V. CONCLUSION

This work aims to create SCIAM (Smart Collaborative IDE with AI Support and Session Management), which will solve some of the most significant issues that currently exist in contemporary software development by incorporating all necessary tools into a single smart environment. Using real-time collaborative coding tools that are also supported by artificial intelligence tools and automated session management tools will increase productivity by reducing redundant work, decreasing coding mistakes, and improving code quality by using artificial intelligence tools that can suggest improvements and identify errors on a real-time basis. Real-time collaboration tools enable collaboration between distributed software developers without causing any interruption in collaboration activities, while automatic saving tools will ensure that the data is not lost under any circumstances. Furthermore, analytics and reporting tools are also incorporated into the development environment that will enable developers and academics to make correct decisions while using the software in both professional and academic situations. Moreover, it facilitates learning in educational institutes using artificial intelligence tools.

REFERENCES

[1] Htoo Htoo Sandi Kyaw, Keiichi Kaneko, Nobuo Funabiki,A Proposal of the Gamification of Programming Learning Assistant System,2024

[2] Vaishali Savale, Vedashri Chaudhari, Tejaswini Wanare, Balamurali Nambiar, Manthan Vasant, Sanskruti Umak, CodeQuest - A Comprehensive Code Learning and Collaboration Platform,2024

[3] Rijul Saini, Gunter Mussbacher,Towards Conflict-Free Collaborative Modelling Using VS Code Extensions,2021

[4] Mingjie Wang, Bicheng Fang, Jinfeng Jiang, Hongfei Fan, Annotation-based Semantic Conflict Prevention in Real Time Collaborative Programming: Approach, Techniques, Prototype, and User Study,2024

[5] Khushwant Virdi, Anup Lal Yadav, Azhar Ashraf Gadoo, Navjot Singh Talwandi,Collaborative Code Editors - Enabling Real-Time Multi User Coding and Knowledge Sharing,2023

[6] Jinfeng Jiang, Yuxiang Xie, Bicheng Fang, Mingjie Wang, Hongfei Fan, Building Temporary Isolated Workspace in Real-Time Collaborative Programming Environment,2023

[7] Mihir Agrawal, Jatin Goyal, Mradul Goyal, Pratham Sukhija, Jaya Sharma, D. Franklin Vinod, CodeXchange: Leaping into the Future of AI-Powered Code Editing,2024

[8] Ankita Amburle, Cheryl Almeida, Oswin Lopes, Nathan Lopes, AI Based Code Error Explainer using Gemini Mode,2024

[9] Lingchen Huang, Huazi Zhang, Rong Li, Yiqun Ge, Jun Wang, AI Coding: Learning to Construct Error Correction Codes,2020

[10] Matteo Ciniselli, Nathan Cooper, Luca Pascarella, Antonio Mastropaolo, Emad Aghajani, Denys Poshyvanyk, Massimiliano Di Penta, Gabriele Bavota, An Empirical Study on the Usage of Transformer Models for Code Completion,2021

[11] Zixiao Zhao, Jing Sun, Cheng-Hao Cai, Zhiyuan Wei, Code Generation Using Self interactive Assistant,2024

[12] Haibin Lai, Peng Peng, Yuhui Lv, Peiwei Cai, Qiongxiong Ma, Zhun Zhang, The Development of a Collaborative Learning Tool for AI Education: Corobolab,2025

[13] Saptaparni Kumar, Haochen Pan, Roger Wang, Lewis Tseng,LiteDoc: Make Collaborative Editing Fast, Scalable, and Robust,2020

[14] Junbang Ma,AI-Driven Web Application Automated Testing and Code Optimization,2025

[15] Abdul Mannan Omi, Monir Hossain, Md Nahidul Islam, Tanni Mittra, Multiple Authors Identification from Source Code Using Deep Learning Model,2021